UAM WSA Reverse Proxy Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAM WSA Reverse Proxy Implementation Guide

This document provides the concept and implementation steps for the AccessMatrix Universal Access Manager (UAM) Web Security Agent (WSA) Reverse Proxy (RP) - a component that intercepts all incoming HTTP requests of Apache HTTP Server. Both the WSA RP (Java) and WSA RP (Native) implementations are covered in this guide.

Intended Reader

WSA Reverse Proxy is normally expected to be configured by Web Access Management (WAM) project team member, i-Sprint Professional Service/partner team member and a representative from Web Server owner.

This document is intended as a reference to carry out the following operations:

- Deployment of WSA RP
- Configuration of WSA RP
- Deployment of WSA RP sample application

Document Scope

This document only provides the necessary steps to deploy and configure the WSA Reverse Proxy. It will not provide documentation on the installation of AccessMatrix system and Apache HTTP Server.

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- *AccessMatrix Common Administration Guide*
- *UAM Administration Guide*

2. Introduction to UAM WSA Reverse Proxy Implementation

This document provides the information on the deployment and implementation of AccesMatrix WSA Reverse Proxy.

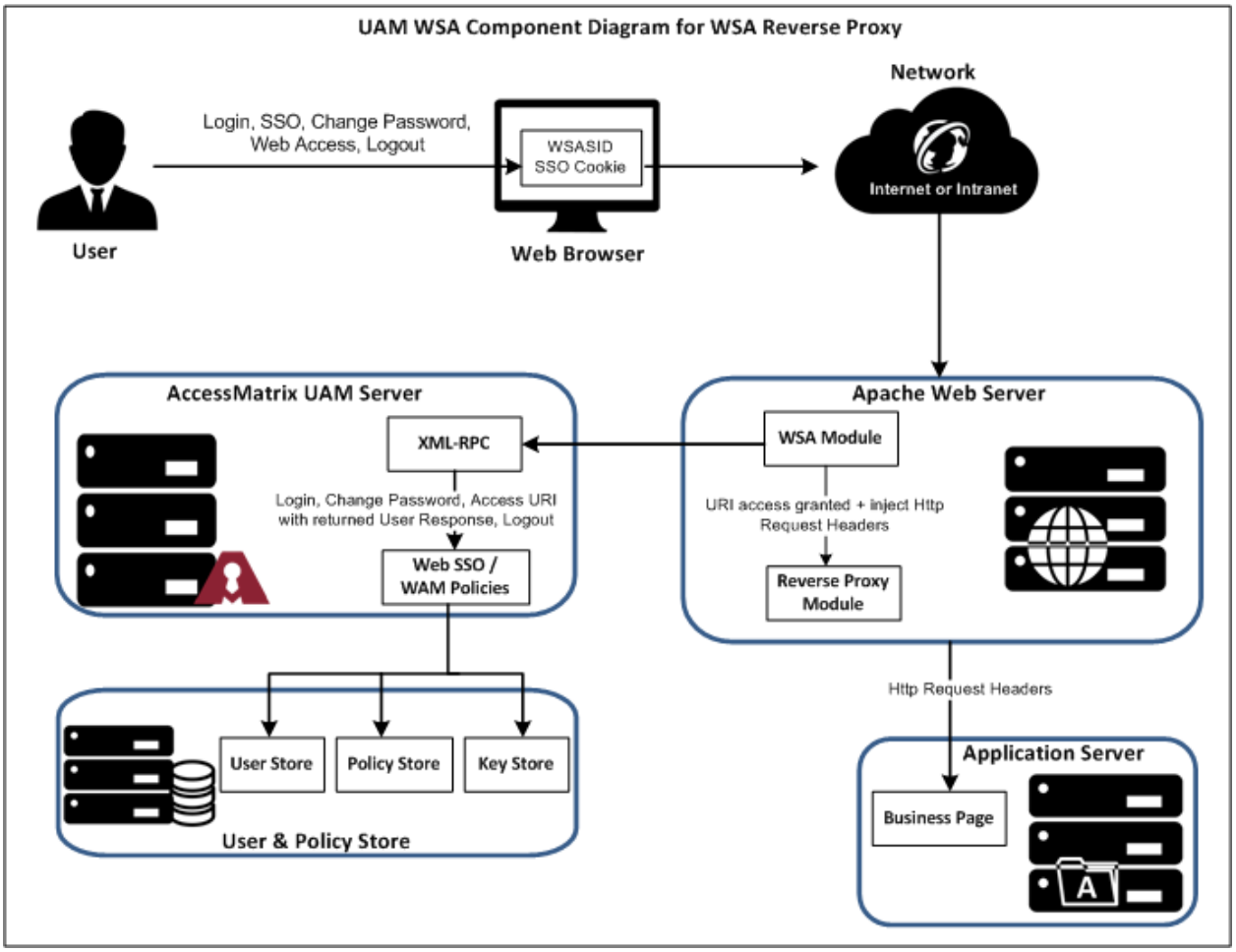
About WSA Reverse Proxy

UAM WSA Reverse Proxy (RP) is an interceptor component that intercepts all incoming HTTP requests of Apache HTTP Server (version 2.4), where the deployment requires a change of network topology such that all application servers are placed behind the reverse proxy web server. This solution only supports deployment on Apache HTTP Server version 2.4 (Red Hat Enterprise Linux (RHEL) 7.4 64-bit).

Advantages of WSA RP compared to integration via standard XML-RPC/SOAP web services API between applications and UAM server:

1. **Transparent Single Sign-On (SSO) session Life Cycle Management**
   SSO session as a cookie is automatically set, refreshed, maintained, and terminated. This saves the application from directly managing the SSO session life cycle.
2. **Performance and Scalability**
   WSA RP has in-memory resource and access cache capability.
3. **Audit Enhancement**
   WSA RP is able to inject user Id into the web server access log that can be analyzed by standard web traffic analysis tools.
4. **Built-in web services and demo pages for quick Proof Of Concept (POC) / prototyping**
5. **Ease of Integration**
   Passing user information, group memberships, roles, etc. via HTTP request header to protected applications.
6. **Extra Web Access Protection**
   Ability to filter out directory traversal and cross-site scripting attacks.
7. **Central and policy-driven configuration via Admin Console**

UAM WSA Component Diagram



The components of the Web Access Management (WAM) solution includes:

1. The **User-Agent (Web Browser)** with cookie support. WSA relies on SSO cookie to maintain state for each SSO session.
2. The **Apache Web Server**.
3. The **WSA Module** that is installed as the first in plugin execution order in the web server.
4. The **Reverse Proxy Module** that is installed as the second in plugin execution order in the web server.
5. The **Application Server**.
6. The **AccessMatrix UAM Derver** where the user, policy and key management are performed. WSA plugin will communicate to the AccessMatrix UAM server via XML-RPC web services.

7. The **AccessMatrix UAM Database**. The user store could be a Relational Database Management System (RDBMS) or Lightweight Directory Access Protocol (LDAP) server. Policy and encryption keys must be managed within an RDBMS.

In a Web SSO/WAM solution, it is necessary for the application to outsource its authentication to the SSO server. Since the application does not directly authenticate the user, there must be a way for the application to retrieve user information from the SSO server. This is normally achieved via:

- WSA module, upon successful authentication, pass the user ID and user information via HTTP request headers (configurable header names) so that it can be picked by Commercial-off-the-shelf (COTS) applications with SSO support or custom applications that use standard servlet/.NET API to retrieve the user ID/information from Http Request Headers.
- Custom applications might be able to integrate with UAM server via XML-RPC, SOAP or RESTful web services.

To simplify the scope, this document assumes application integration with UAM is done using Http Request Headers.

Implementation Checklist

Below is the checklist for implementing UAM WSA Reverse Proxy.

Tasks	Reference
Deploy WSA Reverse Proxy Component	UAM WSA Reverse Proxy Deployment
Configure WSA Reverse Proxy	UAM WSA RP Reverse Proxy Deployment
Deploy Sample WSA Reverse Proxy Application	UAM WSA Reverse Proxy Sample Application

3. UAM WSA Reverse Proxy Deployment

Prior to setting up the WSA Reverse Proxy component, ensure that the following requirements are ready and set up properly:

- A WSA Reverse Proxy server (Red Hat Enterprise Linux (RHEL) v7.x Operating System (OS)).
- Yum package manager must be installed in RHEL server.
- Accessible repository to install Apache HTTP Server 2.4 (offline repository is also available in RHEL 7.x DVD distributions). Refer to the Install Apache HTTP Server section for the installation details.
- Installed Java Virtual Machine (JVM) on the WSA RP server with a minimum version of Java 1.8.
- AccessMatrix UAM Server
- Application Server (to host the Web Application that will be protected by WSA Reverse Proxy)
- Obtain the WSA Reverse Proxy zip file (e.g. **AccessMatrix-5.6.7.6711-GA-wsarp.zip**) and extract the content to a path.

> **Notes**:
>
> - In this guide, *<WSARP_PATH>* is used to refer to the directory of AccessMatrix WSA component, e.g. */etc/httpd/*.
> - *<ACMATRIX_ROOT>* is used to refer to the installation directory of AccessMatrix System, e.g. */opt/acmatrix/tomcat*.

Install Apache HTTP Server

There are many ways to install the Apache HTTP server and one of the common ways is to install it using Yum. Simply run the following command:

```
yum install httpd
```

> **Note**: If installation from the DVD is required, then follow the steps provided here to set up yum repository for locally-mounted DVD on RHEL v7.x.

WSA Reverse Proxy Directory Structure

Refer to the following sections for the directory structure:

- WSA Reverse Proxy (Java) Directory Structure
- WSA Reverse Proxy (Native) Directory Structure

WSA Reverse Proxy (Java) Directory Structure

Below is the directory structures of the WSA Reverse Proxy zip file (e.g. **AccessMatrix-5.6.2.6214-GA-wsarp.zip**).

Folder Name	Subfolder Name	Files	Description
httpd	conf	ehcache.xml	General purpose caching.
		testagent.keystore	Agent keystore containing client certificates representing WSA identity. It is referenced in **wsa.conf**. **Note:** Replace it with the user's own certificate in a production system.
		testtrust.keystore	Trust store containing the Certificate Authority (CA) certificates. It is referenced in **wsa.conf**. **Note:** Replace it with the user's own certificate in a production system.
		wsa.conf	WSA configuration file.
		wsarp.crt	Sample Secure Sockets Layer (SSL) Certificate File for testing purpose (hostname: server.wsa.com).
		wsarp.key	Sample SSL Certificate Key File for testing purpose (hostname: server.wsa.com)
	conf.d	wsarp.conf	WSA Reverse Proxy configuration file.
	conf.modules.d	00-mpm.conf	The Multi-Processing Module (MPM) configuration file.
	jars		Contains the JAR files which are required by WSA Reverse Proxy.
	modules	mod_am_wsa.so	The WSA module file.
sample_wsa_app	wsa		Contains the WSA demo web application files.

WSA Reverse Proxy (Native) Directory Structure

Below is the directory structures of the WSA Reverse Proxy tar.gz file (e.g. **AccessMatrix-5.6.7.6710-GA-wsarp-native.tar.gz**).

Folder Name	Subfolder Name	Files	Description
wsa	auth		Contains the default WSA authentication page.
	conf	testagent.pem	Agent keystore containing client certificates representing WSA identity. It is referenced in **wsarp.conf**. **Note:** Replace it with the user's own certificate in a production system.
		testtrust.pem	Trust store containing the Certificate Authority (CA) certificates. It is referenced in **wsarp.conf**. **Note:** Replace it with the user's own certificate in a production system.
		wsarp.conf	WSA Reverse Proxy configuration file.
		wsarp.crt	Sample Secure Sockets Layer (SSL) Certificate File for testing purpose (hostname: server.wsa.com).
		wsarp.key	Sample SSL Certificate Key File for testing purpose (hostname: server.wsa.com)
	mod_am_wsa.so		The WSA module file.
readme.txt			Contains the setup and installation steps of WSA RP component.
wsatest			Contains the WSA demo web application files.

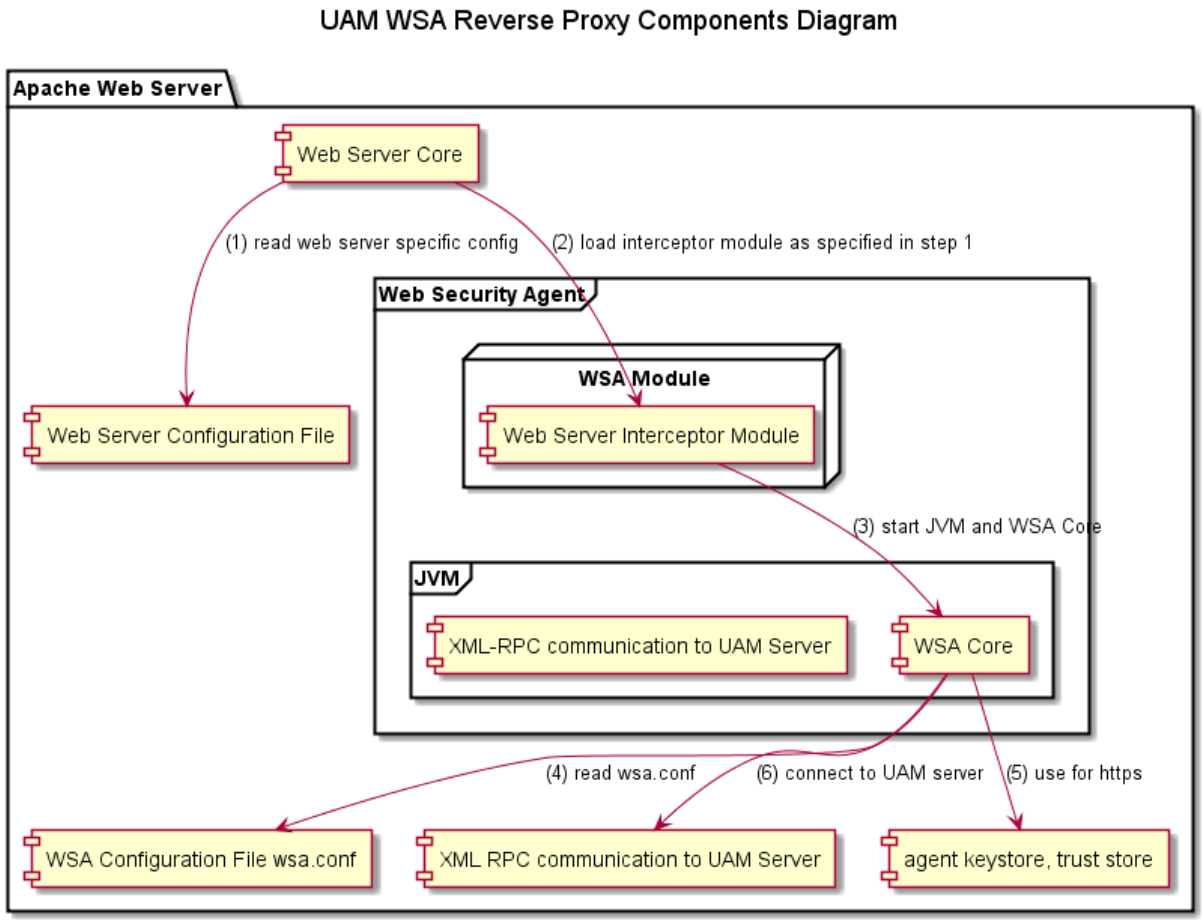
The administrator may proceed to deploy and configure the WSA Reverse Proxy component manually onto the target Apache HTTP server in order to integrate WSA Reverse Proxy component with the Web application server and to enforce access control to the Web applications. This will be discussed in details in the following sections.

UAM WSA Reverse Proxy Components Diagram

Refer to the following for the components diagram:

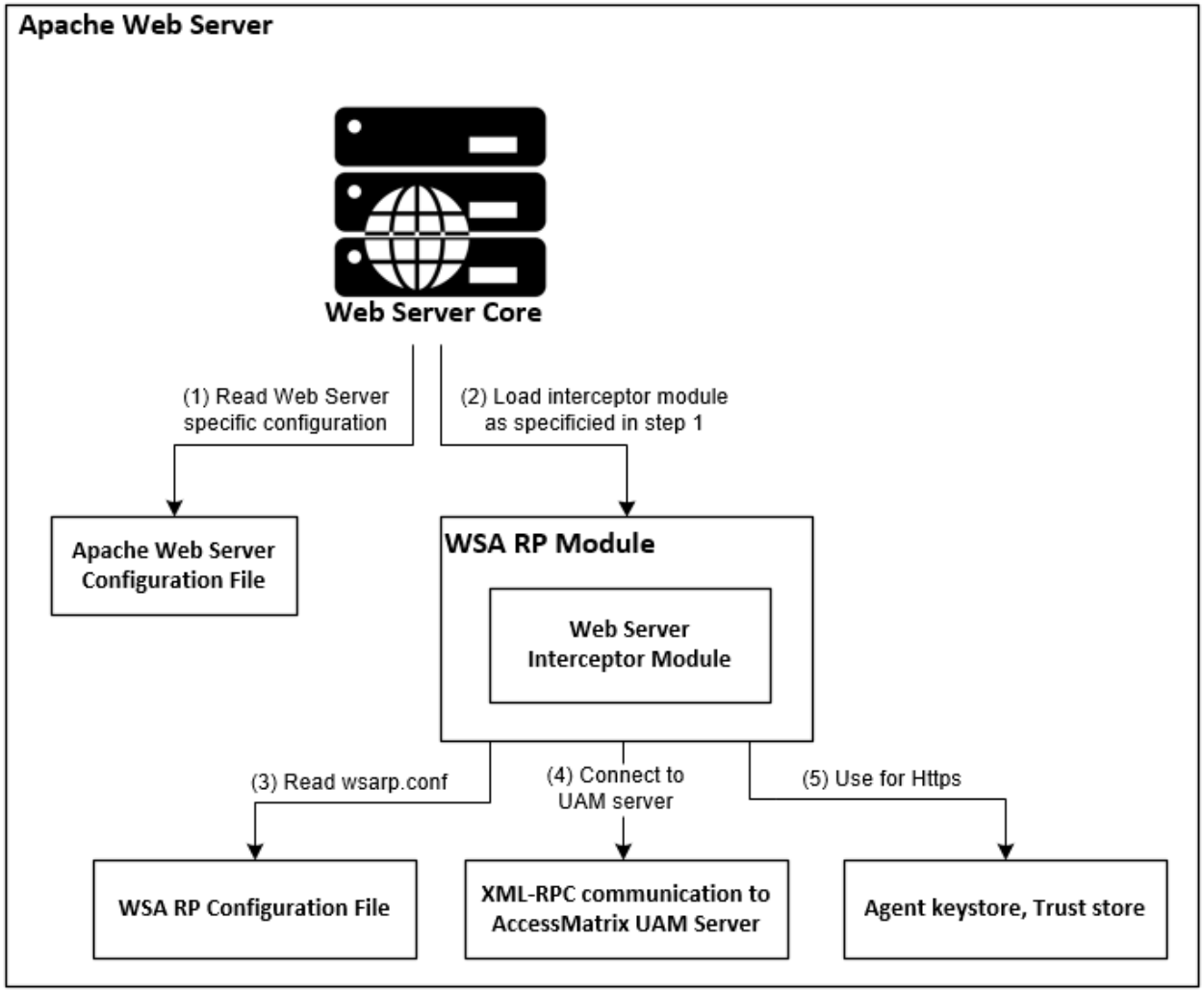
- WSA Reverse Proxy (Java) Components Diagram
- WSA Reverse Proxy (Native) Components Diagram

WSA Reverse Proxy (Java) Components Diagram



WSA Reverse Proxy (Native) Components Diagram

The diagram below illustrates the WSA Reverse Proxy components in Apache Web Server.



Deploy WSA Reverse Proxy

This section describes the steps to deploy WSA Reverse Proxy (RP) module onto Apache HTTP/Web server version 2.4 installed in Red Hat Enterprise Linux version 7.4.

WSA Reverse Proxy (Java)	1. From the extracted WSA RP zip file, copy the content of httpd folder to the Apache HTTP Server version 2.4 home folder (i.e. */etc/httpd/* or represented as <*WSARP_PATH*> in this document).
	2. Open and modify the <*WSARP_PATH*>*/conf.modules.d/00-mpm.conf* file by uncommenting the MPM worker and commenting out the MPM prefork as shown below:
	Code
	`# Select the MPM module which should be used by uncommenting exactly`
	`# one of the following LoadModule lines:`
	
	`# prefork MPM: Implements a non-threaded, pre-forking`

```
web server
# See:
http://httpd.apache.org/docs/2.4/mod/prefork.html
#LoadModule mpm_prefork_module
modules/mod_mpm_prefork.so

# worker MPM: Multi-Processing Module implementing a
hybrid
# multi-threaded multi-process web server
# See: http://httpd.apache.org/docs/2.4/mod/worker.html
#
LoadModule mpm_worker_module modules/mod_mpm_worker.so

# event MPM: A variant of the worker MPM with the goal
of consuming
# threads only for connections with active processing
# See: http://httpd.apache.org/docs/2.4/mod/event.html
#
#LoadModule mpm_event_module modules/mod_mpm_event.so
```

3. Save the changes.
4. Launch a terminal and run the following commands:

```
firewall-cmd --zone=public --add-port=80/tcp --permanent
firewall-cmd --reload
```

> **Note**: This will open the port 80 with protocol tcp in the public zone permanently.

5. Configure the hostnames for WSA RP server, AccessMatrix UAM server, and target application server in its respective hosts files. For example:

Code

```
# WSA Reverse Proxy
192.168.50.100 server.wsa.com
# Access Matrix Server
192.168.1.44 am5.wsa.com
# Sample WSA application
192.168.50.101 app.wsa.com
```

6. Save the changes.
7. You may proceed to configure the WSA RP component as described in the UAM WSA Reverse Proxy Configuration section.
8. Start the Apache HTTP/web server by running any of the following commands:
   - To run Apache as a service:
     `httpd -k start`
   - To run Apache as a daemon:
     - Configure Security-Enhanced Linux (SELinux) to give access for httpd to execute an external process (JVM) and create socket connection in Java:

```
setsebool -P httpd_execmem on
setsebool -P httpd_can_network_connect on
```

- Run this command:
  `apachectl start`

9. Check the log file at *<WSARP_PATH>/logs/error_log*. WSA RP is deployed and started successfully if the following log is displayed and there is no exception returned (example):
   `2021-06-0411:51:45.782 [AM5WSA:INFO ] WSA has started successfully WSA_VERSION=5.6.2.6200`
10. You may stop the Apache HTTP/web server by running any of the following commands:
    - To stop Apache as a service, run this command:
      `httpd -k stop`
    - To stop Apache as a daemon, run this command:
      `apachectl stop`

WSA Reverse Proxy (Native)

1. From the extracted WSA RP tar.gz file, copy the *wsa* folder to the Apache HTTP Server version 2.4 home folder (i.e. */etc/httpd/* or represented as *<WSARP_PATH>* in this document).
2. Open the Apache HTTP server configuration file located in *<WSARP_PATH>/conf/httpd.conf* and add this WSA RP configuration IncludeOptional wsa/conf/wsarp.conf to the Supplemental configuration section as shown below:

   Code

   ```
   # Supplemental configuration
   #
   # Load config files in the "/etc/httpd/conf.d"
   directory, if any.
   IncludeOptional conf.d/*.conf
   IncludeOptional wsa/conf/wsarp.conf
   ```

   Note: This change will include the WSA RP configuration file from within the Apache server configuration files.
3. Save the changes.
4. Open and modify the *<WSARP_PATH>/conf.modules.d/00-mpm.conf* file by uncommenting the MPM worker and commenting out the MPM prefork as shown below:

   Code

   ```
   # Select the MPM module which should be used by
   uncommenting exactly
   # one of the following LoadModule lines:

   # prefork MPM: Implements a non-threaded, pre-forking
   web server
   # See:
   http://httpd.apache.org/docs/2.4/mod/prefork.html
   #LoadModule mpm_prefork_module
   ```

```
modules/mod_mpm_prefork.so

# worker MPM: Multi-Processing Module implementing a
hybrid
# multi-threaded multi-process web server
# See: http://httpd.apache.org/docs/2.4/mod/worker.html
#
LoadModule mpm_worker_module modules/mod_mpm_worker.so

# event MPM: A variant of the worker MPM with the goal
of consuming
# threads only for connections with active processing
# See: http://httpd.apache.org/docs/2.4/mod/event.html
#
 m
#LoadModule mpm_event_module modules/mod_mpm_event.so
```

5. Save the changes.
6. Launch a terminal and run the following commands:

```
firewall-cmd --zone=public --add-port=80/tcp --permanent
firewall-cmd --reload
```

> **Note**: This will open the port 80 with protocol tcp in the public zone permanently.

7. Configure the hostnames for WSA RP server, AccessMatrix UAM server, and target application server in its respective hosts files. For example:

Code

```
# WSA Reverse Proxy
192.168.50.100   server.wsa.com
# Access Matrix Server
192.168.1.44     am5.wsa.com
# Sample WSA application
192.168.50.101   app.wsa.com
```

8. Save the changes.
9. You may proceed to configure the WSA RP component as described in UAM WSA Reverse Proxy Configuration section.
10. Start the Apache HTTP/web server by running any of the following commands:
    - To run Apache as a service:
      `httpd -k start`
    - To run Apache as a daemon:
      a. Configure Security-Enhanced Linux (SELinux) to change the filetype of **mod_am_wsa.so** to become a webserver module and to allow its connection to the network:

```
chcon -t httpd_modules_t /etc/httpd/wsa/mod_am_wsa.so
setsebool -P httpd_can_network_connect on
```

	b. Run this command:
	`apachectl start`
	11. Check the log file at <*WSARP_PATH*>*/logs/error_log*. WSA RP is deployed and started successfully if there is no error in the log and no exception returned.
	12. You may stop the Apache HTTP/web server by running any of the following commands:
	○ To stop Apache as a service, run this command:
	`httpd -k stop`
	○ To stop Apache as a daemon, run this command:
	`apachectl stop`

4. UAM WSA Reverse Proxy Configuration

Refer to the respective implementation checklists for configuring the UAM WSA Reverse Proxy:

- WSA Reverse Proxy (Java) Implementation Checklist
- WSA Reverse Proxy (Native) Implementation Checklist

WSA Reverse Proxy (Java) Implementation Checklist

WSA RP Configuration Files

The table below lists the configurations that must be set up for WSA RP:

Configuration File	Description
WSA Configuration (conf/wsa.conf)	This file contains WSA settings such as AccessMatrix XML-RPC URL, keystore and truststore file configurations, session cache, and SAML realm Id configurations.
WSA RP Configuration (conf.d/wsarp.conf)	This file contains WSA RP settings such as MPM Worker module and virtual host configurations.

WSA RP Configuration Steps

The following sections detail the configuration steps for the basic WSA RP setup:

Prior to WSA Reverse Proxy component configuration, it is assumed that the AccessMatrix UAM Server has been set up properly. Otherwise, the administrator should proceed to set up the AccessMatrix UAM Server first.

Having ensured that the UAM Server, as well as the target web application server, have been installed properly, the administrator may proceed to configure the WSA Reverse Proxy component.

Below is the list of configuration steps for basic WSA RP setup:

1. Configure JVM Path on WSA RP Server
2. Configure the WSA RP Connection to the AccessMatrix Server
3. Configure the SSL Mutual Authentication
4. Configure the Authorization Cache
5. Configure the Session Cache

6. Set Up SSL on Apache Web Server
7. Set Up SSL Between the Apache Web Server and the Target Applications (Optional)
8. Configure the WSA RP Log Properties (Optional)

Optionally, click here to continue the WSA RP deployment.

You may proceed to perform the following advanced steps for additional WSA RP configurations:

1. Configure High Availability
2. Capacity Planning
3. Configure JVM Settings for WSA
4. Support X-Forwarder-For and Forwarded Headers
5. Enable Multiple SSO Proxy Support
6. Configure WSA RP to load Apache MPM Worker
7. Support Load Balancing Using WSA Reverse Proxy
8. Configure SAML SSO to WSA RP
9. Configure CLP URL

Click here to complete the WSA RP deployment.

WSA Reverse Proxy (Native) Implementation Checklist

WSA RP Configuration Files

The table below lists the configurations that must be set up for WSA RP:

Configuration File	Description
WSA RP Configuration (conf/wsarp.conf)	This file contains WSA RP settings such as AccessMatrix XML-RPC URL, keystore and truststore file configurations, virtual host configuration, and MPM Worker module configuration.

The following sections detail the configuration steps for the basic WSA RP setup:

WSA RP Configuration Steps

Prior to WSA Reverse Proxy component configuration, it is assumed that the AccessMatrix UAM Server has been set up properly. Otherwise, the administrator should proceed to set up the AccessMatrix UAM Server first.

Having ensured that the UAM Server, as well as the target web application server, have been installed properly, the administrator may proceed to configure the WSA Reverse Proxy component.

Below is the list of configuration steps for basic WSA RP setup:

1. Configure the WSA RP Connection to the AccessMatrix Server
2. Configure the SSL Mutual Authentication
3. Configure the Session Cache
4. Set Up SSL on Apache Web Server
5. Set Up SSL Between the Apache Web Server and the Target Applications (Optional)
6. Configure the WSA RP Log Properties (Optional)

Optionally, click here to continue the WSA RP deployment.

You may proceed to perform the following advanced steps for additional WSA RP configurations:

1. Enable WSA RP's Default Authentication Pages (Optional)
2. Configure High Availability
3. Capacity Planning
4. Support X-Forwarder-For and Forwarded Headers
5. Configure WSA RP to Load Apache MPM Worker
6. Support Load Balancing Using WSA Reverse Proxy
7. Configure SAML SSO to WSA RP
8. Configure CLP URL

Click here to complete the WSA RP deployment.

4.1. WSA RP Configurations

The following pages provide detailed information on configuring the UAM WSA Reverse Proxy:

- WSA RP (Java) Configurations
- WSA RP (Native) Configurations

4.1.1. WSA RP (Java) Configurations

Configure JVM Path on WSA RP Server

It is required to change the default JvmPath value to the actual JDK installation directory. To configure:

1. Navigate to <*WSARP_PATH*>*/conf.d/* and open **wsarp.conf**.
2. Modify the JvmPath attribute with the actual JDK version installed as shown below:

```
Code

  <IfModule wsa_module>
  ...
  JvmPath "/usr/lib/jvm/java-1.8.0-openjdk-1.8.0.131-
11.b12.el7.x86_64/jre/lib/amd64/server/libjvm.so"
  ...
</IfModule>
```

   - JvmPath - The path to the JVM library file.
3. Save the changes.

Configure WSA RP to Connect to the AccessMatrix Server

WSA Reverse Proxy needs to communicate with the AccessMatrix UAM server via XML-RPC web service. A configuration file is required to specify the communication properties, logging and caching properties unique for each WSA Reverse Proxy. The configuration below establishes the connection between WSA RP and AccessMatrix server. To configure:

1. Navigate to <*WSARP_PATH*>*/conf/* and open **wsa.conf**. For a web server module, the **wsa.conf** file is usually automatically loaded as long as the <*WSARP_PATH*> is known.
2. Look for the host parameter and modify the value to refer to the relevant AccessMatrix servers as shown below:

```
Code

<access_matrix version="5.6.2">
    <wsa id="usoagent">
      <server>
        <authed>
          <host name="SG"
url="http://am5sg1.wsa.com:8080/am5/xmlrpc"/>
          ...
```

```
      </authed>
    </server>
```

- Change the XML-RPC URL setting for the AccessMatrix server (shown above) to point to the AccessMatrix server's host URL:
  - For SSL Server Connection (a mandatory configuration for Production environment):
    "https://<AM_Hostname>:<AM_Secured_Port>/am5/xmlrpc"
  - For Non-SSL Server Connection:
    "http://<AM_Hostname>:<AM_Non-Secured_Port>/am5/xmlrpc"
- If SSL server connection is used where the Tomcat bundled server certificate is used for establishing the SSL connection between WSA Reverse Proxy and AccessMatrix server, the **host** URL should be set to the following value:
  "https://testserver:<AM_Secured_Port>/am5/xmlrpc"
  Example: "https://testserver:8443/am5/xmlrpc"
  Since the default Apache Tomcat web server SSL certificate (bundled with AccessMatrix server) uses "testserver" as its server hostname, https connection will require the use of this hostname. In production, the user should generate their own server certificate with proper hostname for https communication. For SIT or UAT environment, the IP address can be mapped with "testserver" in the Windows host file or */etc/hosts* in Unix.
- If non-SSL connection is used, by default, a simulated certificate login will be used to represent the WSA Reverse Proxy identity to the UAM server. This will simulate the client certification authentication for POC or quick test without the need to set up actual client cert authentication. To disallow simulated certificate login, comment out the following from **amsystem.properties** within the UAM server *WEB-INF/classes/*:

Properties (.properties files)

```
 # == XML-RPC ==
com.isprint.am.server.xmlrpc.XmlRpcServlet.$simCert.file=conf/
testagent.cer
```

Note: You are discouraged from using the simulated certificate in a production environment as it has lower security compared to actual login using a client certificate.

3. You may also set the connect timeout parameter to set how long WSA RP will attempt to establish an HTTP connection with the AccessMatrix server as shown below:

```
Code

 <access_matrix version="5.6.2">
    <wsa id="usoagent">
      <server>
        <authed>
          ...
          <connect timeout="30"/>
        <authed>
      ...
  </access_matrix>
```

The default value is 30 seconds.

4. Save the changes.

Configure SSL Mutual Authentication Between WSA RP Module and AccessMatrix Server

By default, when HTTPS is used for connection to the AccessMatrix server, WSA RP will use the i-Sprint generated client certificate located in **testagent.keystore** and self-signed CA certificate in "testtrust.keystore" that are compatible with the Tomcat bundled in AccessMatrix UAM server and Certificate Authority (CA) certificate. In order to use your own certificate for SSL mutual authentication between WSA RP and AccessMatrix server, especially in production environment, then you must change the agent keystore and trust store to your own.

To configure the keystore and truststore settings:

1. Navigate to *<WSARP_PATH>/conf/* and open **wsa.conf**, if not opened yet.
2. Look for the keystore filename and truststore parameters and modify the values, as shown below:

```
Code

  <access_matrix version="5.6.2">
      ...
      <keystore
filename="<WSARP_PATH>/conf/testagent.keystore"
password="passphrase"
truststore="<WSARP_PATH>/conf/testtrust.keystore"
truststorePassword="passphrase"/>
      ...
  </access_matrix>
```

3. Save the changes.

If the selected connection type is SSL, WSA Reverse Proxy needs to load its private key and X.509 digital certificate from the keystore file.

Notes:

- Only Java Key Store (JKS) format is supported.
- Keystore file is a JKS file containing both the WSARP certificate and private key. This WSARP certificate is used as the client-side certificate when establishing mutual authenticated SSL. AccessMatrix Server's SSL connector's truststore must contain this WSARP certificate or its CA certificate.
- Truststore file contains the AccessMatrix Server's SSL certificate or its CA certificate.

Configure Authorization Cache

WSA Reverse Proxy has a local session-based authorization cache attributes which are configurable. To configure:

1. Navigate to *<WSARP_PATH>/conf/* and open **wsa.conf**.
2. Look for the authz_cache parameter and modify the value as shown below:

```
Code

<authz_cache maxElementsInMemory="10000"
timeToLiveSeconds="600" timeToIdleSeconds="300"/>
```

   - maxElementsInMemory - Sets the number of elements that may be stored in the defined cache. The default value is 10000.
   - timeToLiveSeconds - Sets the maximum time between creation time and when an element expires. The default value is 600.
   - timeToIdleSeconds - Sets the maximum amount of time between accesses before an element expires. The default value is 300.
3. Save the changes.

Configure Session Cache

In WSA RP configuration file, you can also set the window period to make WSA check or refresh the session with AccessMatrix server periodically. By default this setting is disabled.

To configure:

1. Navigate to *<WSARP_PATH>/conf/* and open **wsa.conf**.
2. Look for the session_cache element, uncomment to enable the feature and modify the value as shown below:

```
Code

  <wsa id="usoagent">
      ...
      <session_cache enabled="on"
sessionRefreshWindowSecs="300" />
      ...
  </wsa>
```

   The attributes are:

   - enabled - toggle on and off the session refresh.
   - sessionRefreshWindowSecs - specifies the time window. After which, it is required to do a server-side refresh of the session even if the session is up to date in terms of last access time. The purpose is to check against the server for cases of persistent sessions that may have been invalidated remotely.
3. Save the changes.

Set Up SSL on Apache Web Server

To enable SSL on Apache Web Server, follow the steps below:

1. Install the SSL module by typing the following Linux command:

   ```
   yum -y install mod_ssl
   ```
2. Add port 443 to be accessed from public by typing the commands below:

   ```
   firewall-cmd --zone=public --add-port=443/tcp --permanent
   firewall-cmd --reload
   ```
3. Open the WSA RP configuration file, i.e. *<WSARP_PATH>/conf.d/wsarp.conf*.
4. Activate the SSL configuration by adding the SSL-related elements inside the <VirtualHost> tag:

```
Code
```

```
LoadModule ssl_module modules/mod_ssl.so
    ...

    <VirtualHost server.wsa.com:443>
      ...

      SSLEngine on
      SSLCipherSuite
HIGH:MEDIUM:!aNULL:!MD5:!SEED:!IDEA
      SSLCertificateFile /etc/httpd/conf/wsarp.crt
      SSLCertificateKeyFile /etc/httpd/conf/wsarp.key
</VirtualHost>
```

- SSLEngine - set to "on" to enable SSL protocol engine usage.
- SSLCipherSuite - specifies the Cipher Suite available for negotiation in SSL handshake.
- SSLCertificateFile - points to a file with certificate data in PEM format.
- SSLCertificateKeyFile - points to the PEM-encoded privated key file for the server.

5. Save the changes.

Additional reference: http://httpd.apache.org/docs/2.4/mod/mod_ssl.html

Set Up SSL between Apache Web Server and Backend Applications (Optional)

If the Apache Web Server needs to communicate with the target application via SSL and the target application is not using a certificate signed by a proper CA, then the communication will fail. To resolve this issue, either of the following solution is recommended:

- Disable the server certificate checking
- Add the certificate to the trusted cert of Apache Web Server Reverse Proxy

> **Note**: Ensure that the target application in Admin Console is configured using the correct SSL port of the web server, e.g. 8443 as shown in the sample configurations. You can configure this in **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.

Disable the Server Certificate Checking

If this is acceptable for the deployment, then perform the following steps:

1. Navigate to *<WSARP_PATH>/conf.d/* and **open wsarp.conf**, if not opened yet.
2. Add the following configuration to disable server certificate checking:

Code

```
SSLProxyEngine on
SSLProxyCheckPeerCN off
SSLProxyCheckPeerName off
SSLProxyCheckPeerExpire off
```

- SSLProxyEngine - set to "on" to enable SSL for proxy usage.
- SSLProxyCheckPeerCN - set to "off" to disable checking of the remote server certificate's CN field.
- SSLProxyCheckPeerName - set to "off" to disable host name checking for remote server certificates.
- SSLProxyCheckPeerExpire - set to "off" to disable checking if remote server certificate is expired.

3. Modify the configuration in the <LocationMatch> tag. Update the context path, URL and port of the target application. Ensure that it is configured using the correct SSL port and protocol as shown in the sample configuration.
4. Save the changes.

Below is an example of the **wsarp.conf** configurations detailed in this section:

Code

```
...

<VirtualHost server.wsa.com:443>
    ServerName server.wsa.com
    ...
    <LocationMatch "/wsatest">
        ProxyPass https://app.wsa.com:8443/wsatest
        ProxyPassReverse
https://app.wsa.com:8443/wsatest
    </LocationMatch>
    ...

    #SSL for target app connection with disabled server
certificate checking
    SSLProxyEngine on
    SSLProxyCheckPeerCN off
    SSLProxyCheckPeerName off
    SSLProxyCheckPeerExpire off

    #SSL for user/browser connection
    SSLEngine on
    SSLCipherSuite HIGH:MEDIUM:!aNULL:!MD5:!SEED:!IDEA
```

```
    SSLCertificateFile /etc/httpd/conf/wsarp.crt
    SSLCertificateKeyFile /etc/httpd/conf/wsarp.key

</VirtualHost>
```

Reference: https://httpd.apache.org/docs/2.4/mod/mod_ssl.html#sslproxycheckpeercn

Error References:

- If SSLProxyEngine is off or not configured, the following error will be displayed:

```
[Tue Jun 0811:39:38.5429942021] [ssl:error] [pid 56539:tid 140167830505216] [remote 192.168.1.44:8443] AH01961: SSL Proxy requested for server.wsa.com:80 but not enabled [Hint: SSLProxyEngine]
[Tue Jun 0811:39:38.5430192021] [proxy:error] [pid 56539:tid 140167830505216] AH00961: HTTPS: failed to enable ssl support for192.168.1.44:443 (app.wsa.com)
```

- If SSLProxyCheckPeerCN, SSLProxyCheckPeerName, and SSLProxyCheckPeerExpire are "off" or not configured, the following error will be displayed:

```
[Tue Jun 0811:42:11.9640132021] [proxy:error] [pid 56722:tid 140537943865088] (502)Unknown error 502: [client 192.168.197.1:1028] AH01084: pass request body failed to 192.168.1.44:443 (app.wsa.com)
[Tue Jun 0811:42:11.9642392021] [proxy:error] [pid 56722:tid 140537943865088] [client 192.168.197.1:1028] AH00898: Error during SSL Handshake with remote server returned by /private/
[Tue Jun 0811:42:11.9642442021] [proxy_http:err
or] [pid 56722:tid 140537943865088] [client 192.168.197.1:1028] AH01097: pass request body failed to 192.168.1.44:443 (app.wsa.com) from 192.168.197.1 ()
```

Add the Certificate to the Trusted Cert of Apache Web Server Reverse Proxy

1. Navigate to <*WSARP_PATH*>*/conf.d/* and open **wsarp.conf**, if not opened yet.
2. Add the following configuration to add the certificate to the trusted cert:

Code

```
SSLProxyEngine on
SSLProxyVerify require
SSLProxyCACertificateFile conf/testtrust.pem
```

   - SSLProxyEngine - set to "on" to enable SSL for proxy usage.
   - SSLProxyVerify - set to "require" to enforce verification of a valid certificate.
   - SSLProxyCACertificateFile - points to the concatenated PEM-encoded CA certificates for remote server authentication.

3. Modify the configuration in the <LocationMatch> tag. Update the context path, URL and port of the target application. Ensure that it is configured using the correct SSL port and protocol as shown in the sample configuration.
4. Save the changes.

Below is an example of the **wsarp.conf** configurations detailed in this section:

Code

```
...

<VirtualHost server.wsa.com:443>
    ServerName server.wsa.com
    ...
    <LocationMatch "/wsatest">
        ProxyPass https://app.wsa.com:8443/wsatest
        ProxyPassReverse
https://app.wsa.com:8443/wsatest
    </LocationMatch>

    #SSL for target app connection with server
certificate added to the trusted cert of Apache Web
Server Reverse Proxy
    SSLProxyEngine on
    SSLProxyVerify require
    SSLProxyCACertificateFile conf/testtrust.pem

    #SSL for user/browser connection
    SSLEngine on
    SSLCipherSuite HIGH:MEDIUM:!aNULL:!MD5:!SEED:!IDEA
    SSLCertificateFile /etc/httpd/conf/wsarp.crt
    SSLCertificateKeyFile /etc/httpd/conf/wsarp.key

</VirtualHost>
```

Reference: https://httpd.apache.org/docs/2.4/mod/mod_ssl.html#sslproxyverify

Error Reference:

- If SSLProxyEngine is "off" or not configured, the following error will be displayed:

```
[Mon Jun 1414:34:33.4023022021] [ssl:error] [pid 25320:tid 140377986168576] [remote
192.168.1.44:8443] AH02039: Certificate Verification: Error (18): self signed certificate
[Mon Jun 1414:34:33.4025252021] [proxy_http:error] [pid 25320:tid 140377986168576]
(103)Software caused connection abort: [client 192.168.197.1:25653] AH01102: error reading status
line from remote server testserver:8443
```

Configure WSA RP Log Properties (Optional)

WSA RP uses the error_log from Apache as the default log file. This log file consolidates the logging from WSA module, WSA Proxy, and other Apache modules. The location of the error log file can be changed in **httpd.conf** by modifying the ErrorLog directive:

```
Code

   ErrorLog "logs/error_log"
```

The logs for Apache and Java modules are configured separately. Logs for Apache module are configured in **wsarp.conf**, while the logs for Java module are configured in **log4j.properties**. However, both will use the error_log file as the log output file.

Apache Module

The number of messages logged to the error_log can be controlled by setting the LogLevel. Perform the following steps to configure the LogLevel in WSA RP:

1. Navigate to *<WSARP_PATH>/conf.d/* and open **wsarp.conf**.
2. Modify the LogLevel directive as shown in the example below:

   ```
   Code

       LogLevel error wsa_module:debug
   ```

   This configuration follows the log module format <module>:<level>. In this example, the debug level is set for wsa_module and an error level is set for other modules. The possible LogLevel values include:

   - debug
   - info
   - notice
   - warn
   - error
   - crit
   - alert
   - emerg
3. Save the changes.

Java Module

WSA RP uses Apache Log4j as the logging utility. To configure:

1. Navigate to <*WSARP_PATH*>*/jars/* and open **log4j.properties**.
2. Modify the log level, if necessary:

```
Code

    log4j.logger.com.isprint.am.wsa=DEBUG, stderr
```

3. Save the changes.

4.1.2. WSA RP (Native) Configurations

Configure WSA RP to Connect to the AccessMatrix Server

WSA Reverse Proxy needs to communicate with the AccessMatrix UAM server via XML-RPC web service. A configuration file is required to specify the communication properties, logging and caching properties unique for each WSA Reverse Proxy. The configuration below establishes the connection between WSA RP and AccessMatrix server. To configure:

1. Navigate to <*WSARP_PATH*>*/wsa/conf/* and open **wsarp.conf**. For a web server module, the **wsarp.conf** file is usually automatically loaded as long as the <*WSARP_PATH*> is known.
2. Look for the Host element and modify the value to refer to the relevant AccessMatrix server as shown below:

```
Code

<IfModule wsa_module>
    Host "SG=http://am5.wsa.com:8080/am5/xmlrpc"
    ...
</IfModule>
```

   - Change the XML-RPC URL setting for the AccessMatrix server (shown above) to point to the AccessMatrix server's host URL:
     - For SSL Server Connection (a mandatory configuration for Production environment):
       "https://<AM_Hostname>:<AM_Secured_Port>/am5/xmlrpc"
     - For Non-SSL Server Connection:
       "http://<AM_Hostname>:<AM_Non-Secured_Port>/am5/xmlrpc"
   - If SSL server connection is used where the Tomcat bundled server certificate is used for establishing the SSL connection between WSA Reverse Proxy and AccessMatrix server, the Host URL should be set to the following value:
     "https://testserver:<AM_Secured_Port>/am5/xmlrpc"
     Example: "https://testserver:8443/am5/xmlrpc"
     Since the default Apache Tomcat web server SSL certificate (bundled with AccessMatrix server) uses "testserver" as its server hostname, https connection will require the use of this hostname. In production, the user should generate their own server certificate with

proper hostname for https communication. For SIT or UAT environment, the IP address can be mapped with "testserver" in the Windows host file or */etc/hosts* in Unix.

- If non-SSL connection is used, by default, a simulated certificate login will be used to represent the WSA Reverse Proxy identity to the UAM server. This will simulate the client certification authentication for POC or quick test without the need to set up actual client cert authentication. To disallow simulated certificate login, comment out the following from **amsystem.properties** within the UAM server *WEB-INF/classes/*:

Properties (.properties files)

```
 # == XML-RPC ==
com.isprint.am.server.xmlrpc.XmlRpcServlet.$simCert.file=conf/testagent.cer
```

> **Note**: You are discouraged from using the simulated certificate in a production environment as it has lower security compared to actual login using a client certificate.

3. You may also set the ConnectionTimeout element to set how long WSA RP will attempt to establish an HTTP connection with the AccessMatrix server as shown below:

Code

```
<IfModule wsa_module>
    ...
    ConnectionTimeout 20
    ...
</IfModule>
```

The default value is 20 seconds.

4. Save the changes.

Configure SSL Mutual Authentication Between WSA RP Module and AccessMatrix Server

By default, when HTTPS is used for connection to the AccessMatrix server, WSA RP will use the i-Sprint generated client certificate located in **testagent.pem** and self-signed CA certificate in **testtrust.pem** that are compatible with the Tomcat bundled in UAM server and Certificate Authority (CA) certificate. In order to use your own certificate for SSL mutual authentication between WSA RP and AccessMatrix server, especially in production environment, then you must change the agent keystore and trust store to your own.

To configure the keystore and truststore settings:

1. Navigate to *<WSARP_PATH>/wsa/conf/* and open **wsarp.conf**, if not opened yet.
2. Look for the KeystoreFile and TruststoreFile elements and modify the values as shown below:

```
Code

<IfModule wsa_module>
    ...
    KeystoreFile wsa/conf/testagent.pem
    TruststoreFile wsa/conf/testtrust.pem
    ...
</IfModule>
```

3. Save the changes.

If the selected connection type is SSL, WSA Reverse Proxy needs to load its private key and X.509 digital certificate from the keystore file.

Notes:

- Only PEM format is supported.
- KeystoreFile is a PEM file containing both the WSARP certificate and private key. This WSARP certificate is used as the client-side certificate when establishing mutual authenticated SSL. AccessMatrix Server's SSL connector's truststore must contain this WSARP certificate or its CA certificate.
- TruststoreFile is a concatenated PEM-encoded CA certificates file. This file must contain the AccessMatrix Server's SSL certificate or its CA certificate.

WSA RP Authorization Cache

WSA Reverse Proxy has a local session-based authorization cache, but does not require configuration.

Configure Session Cache

In WSA RP configuration file, you can also set the window period to make WSA check or refresh the session with AccessMatrix server periodically.

To configure:

1. Navigate to *<WSARP_PATH>/wsa/conf/* and open **wsarp.conf**, if not opened yet.
2. Look for the SessionRefreshWindowSecs element and modify the value as shown below:

Code

```
<IfModule wsa_module>
    ...
    SessionRefreshWindowSecs 300
    ...
</IfModule>
```

SessionRefreshWindowSecs sets the time window to do a server-side refresh of the session even if the session is up to date in terms of the last access time. The purpose is to check against the server for cases of persistent sessions that may have been invalidated remotely. The default value is 300 seconds. Setting to "0" means local verify/refresh session is disabled.

3. Save the changes.

Set Up SSL on Apache Web Server

To enable SSL on Apache Web Server, follow the steps below:

1. Install the SSL module by typing the following Linux command:

   ```
   yum -y install mod_ssl
   ```

2. Add port 443 to be accessed from public by typing the commands below:

   ```
   firewall-cmd --zone=public --add-port=443/tcp --permanent
   firewall-cmd --reload
   ```

3. Open the WSA RP configuration file, i.e. <*WSARP_PATH*>*/wsa/conf/wsarp.conf*.

4. Activate the SSL configuration by uncommenting the following section in the configuration file:

   Code

   ```
   <VirtualHost *:443>
       ServerName ${wsaServerName}
       ...
       <LocationMatch "/wsatest/">
           ProxyPass ${wsaApp}
           ProxyPassReverse ${wsaApp}
       </LocationMatch>
       SSLEngine on
       SSLCipherSuite HIGH:MEDIUM:!aNULL:!MD5:!SEED:!IDEA
       SSLCertificateFile wsa/conf/wsarp.crt
       SSLCertificateKeyFile wsa/conf/wsarp.key
   </VirtualHost>
   ```

   - SSLEngine - set to "on" to enable SSL protocol engine usage.
   - SSLCipherSuite - specifies the Cipher Suite available for negotiation in SSL handshake.
   - SSLCertificateFile - points to a file with certificate data in PEM format.

- SSLCertificateKeyFile - points to the PEM-encoded privated key file for the server.

5. Save the changes.

Additional Reference:

http://httpd.apache.org/docs/2.4/mod/mod_ssl.html

Set Up SSL between Apache Web Server and Target Applications (Optional)

If the Apache Web Server needs to communicate with the target application via SSL and the target application is not using a certificate signed by a proper CA, then the communication will fail. To resolve this issue, either of the following solution is recommended:

- Disable the server certificate checking
- Add the certificate to the trusted cert of Apache Web Server Reverse Proxy

> **Note**: Ensure that the target application in Admin Console is configured using the correct SSL port of the web server, e.g. 8443, as shown in the sample configurations. You can configure this in **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.

Disable the Server Certificate Checking

If this is acceptable for the deployment, then perform the following steps:

1. Navigate to *<WSARP_PATH>/wsa/conf/* and open **wsarp.conf**, if not opened yet.
2. Add the following configuration to disable server certificate checking:

```
SSLProxyEngine on
SSLProxyCheckPeerCN off
SSLProxyCheckPeerName off
SSLProxyCheckPeerExpire off
```

   - SSLProxyEngine - set to "on" to enable SSL for proxy usage.
   - SSLProxyCheckPeerCN - set to "off" to disable checking of the remote server certificate's CN field.
   - SSLProxyCheckPeerName - set to "off" to disable host name checking for remote server certificates.

- SSLProxyCheckPeerExpire - set to "off" to disable checking if remote server certificate is expired.

3. Modify the configuration in the <LocationMatch> tag. Update the context path, URL and port of the target application. Ensure that it is configured using the correct SSL port and protocol as shown in the sample configuration.
4. Save the changes.

Below is an example of the **wsarp.conf** configurations detailed in this section:

Code

```
...
Define wsaApp http://app.wsa.com/wsatest/
Define wsaServerName server.wsa.com
...

<VirtualHost *:443>
    ServerName ${wsaServerName}
    ...
    <LocationMatch "/wsatest/">
        ProxyPass ${wsaApp}
        ProxyPassReverse ${wsaApp}
    </LocationMatch>

    #SSL for target app connection with disabled server
certificate checking
    SSLProxyEngine on
    SSLProxyCheckPeerCN off
    SSLProxyCheckPeerName off
    SSLProxyCheckPeerExpire off

    #SSL for user/browser connection
    SSLEngine on
    SSLCipherSuite HIGH:MEDIUM:!aNULL:!MD5:!SEED:!IDEA
    SSLCertificateFile wsa/conf/wsarp.crt
    SSLCertificateKeyFile wsa/conf/wsarp.key
</VirtualHost>
```

Reference: https://httpd.apache.org/docs/2.4/mod/mod_ssl.html#sslproxycheckpeercn

Error References:

- If SSLProxyEngine is off or not configured, the following error will be displayed:

  [Tue Jun 08 11:39:38.542994 2021] [ssl:error] [pid 56539:tid 140167830505216] [remote 192.168.1.44:8443] AH01961: SSL Proxy requested for server.wsa.com:80 but not enabled [Hint: SSLProxyEngine]

```
[TueJun0811:39:38.5430192021][proxy:error][pid56539:tid140167830505216]AH00961:HTTPS:fail
edtoenablesslsupportfor192.168.1.44:443(app.wsa.com)
```

- If SSLProxyCheckPeerCN, SSLProxyCheckPeerName, and SSLProxyCheckPeerExpire are "off" or not configured, the following error will be displayed:

```
[Tue Jun 0811:42:11.9640132021] [proxy:error] [pid 56722:tid 140537943865088] (502)Unknown
error 502: [client 192.168.197.1:1028] AH01084: pass request body failed to 192.168.1.44:443
(app.wsa.com)
[Tue Jun 0811:42:11.9642392021] [proxy:error] [pid 56722:tid 140537943865088] [client
192.168.197.1:1028] AH00898: Error during SSL Handshake with remote server returned by
/private/
[Tue Jun 0811:42:11.9642442021] [proxy_http:err
or] [pid 56722:tid 140537943865088] [client 192.168.197.1:1028] AH01097: pass request body
failed to 192.168.1.44:443 (app.wsa.com) from 192.168.197.1 ()
```

Add the Certificate to the Trusted Cert of Apache Web Server Reverse Proxy

1. Navigate to <*WSARP_PATH*>*/wsa/conf/* and open **wsarp.conf**, if not opened yet.
2. Add the following configuration to add the certificate to the trusted cert:

```
Code

SSLProxyEngine on
    SSLProxyVerify require
    SSLProxyCACertificateFile conf/testtrust.pem
</VirtualHost>
```

   - SSLProxyEngine - set to "on" to enable SSL for proxy usage.
   - SSLProxyVerify - set to "require" to enforce verification of a valid certificate.
   - SSLProxyCACertificateFile - points to the concatenated PEM-encoded CA certificates for remote server authentication.

3. Modify the configuration in the <LocationMatch> tag. Update the context path, URL and port of the target application. Ensure that it is configured using the correct SSL port and protocol as shown in the sample configuration.
4. Save the changes.

Below is an example of the **wsarp.conf** configurations detailed in this section:

```
Code

...
Define wsaApp http://app.wsa.com/wsatest/
Define wsaServerName server.wsa.com
```

```
...

<VirtualHost *:443>
    ServerName ${wsaServerName}
    ...
    <LocationMatch "/wsatest/">
        ProxyPass ${wsaApp}
        ProxyPassReverse ${wsaApp}
    </LocationMatch>

    #SSL for target app connection with server
certificate added to the trusted cert of Apache Web
Server Reverse Proxy
    SSLProxyEngine on
    SSLProxyVerify require
    SSLProxyCACertificateFile conf/testtrust.pem

    #SSL for user/browser connection
    SSLEngine on
    SSLCipherSuite HIGH:MEDIUM:!aNULL:!MD5:!SEED:!IDEA
    SSLCertificateFile wsa/conf/wsarp.crt
    SSLCertificateKeyFile wsa/conf/wsarp.key

</VirtualHost>
```

Reference: https://httpd.apache.org/docs/2.4/mod/mod_ssl.html#sslproxyverify

Error Reference:

- If SSLProxyEngine is off or not configured, the following error will be displayed:

  [Mon Jun 1414:34:33.4023022021] [ssl:error] [pid 25320:tid 140377986168576] [remote 192.168.1.44:8443] AH02039: Certificate Verification: Error (18): self signed certificate
  [Mon Jun 1414:34:33.4025252021] [proxy_http:error] [pid 25320:tid 140377986168576] (103)Software caused connection abort: [client 192.168.197.1:25653] AH01102: error reading status line from remote server testserver:8443

Configure WSA Log Properties (Optional)

WSA RP uses the error_log from Apache as the default log file. This log file consolidates the logging from WSA module, WSA Proxy, and other Apache modules. The location of the error log file can be changed in **httpd.conf** by modifying the ErrorLog directive:

Code

```
   ErrorLog "logs/error_log"
```

The number of messages logged to the error_log can be controlled by setting the LogLevel. Perform the following steps to configure the LogLevel in WSA RP:

1. Navigate to *<WSARP_PATH>/conf.d/* and open **wsarp.conf**.
2. Modify the LogLevel directive as shown in the example below:

   Code

   ```
   LogLevel error wsa_module:debug
   ```

   This configuration follows the log module format <module>:<level>. In this example, the debug level is set for wsa_module and an error level is set for other modules. The possible LogLevel values include:

   - debug
   - info
   - notice
   - warn
   - error
   - crit
   - alert
   - emerg
3. Save the changes.

4.2. Advanced Configurations

Advanced features are available to cater for individual organizations' needs. It is generally not required to configure these features.

The following pages provide detailed information on configuring these advanced features:

- WSA RP (Java) Advanced Configurations
- WSA RP (Native) Advanced Configurations

4.2.1. WSA RP (Java) Advanced Configurations

Advanced features are available to cater for individual organizations' needs. It is generally not required to configure these features.

Configure High Availability

This section describes the specific steps in configuring the WSA module to support automatic server failover.

WSA supports the following high availability options:

- Built-in software failover
- Standard web load balancer (recommended if there are more than 2 AccessMatrix servers)

When using the built-in software failover, if the first AccessMatrix server in the list is not available, the access request will be automatically retried on the next available host. Access request will be automatically redirected back to the original AccessMatrix server when it is detected to be available again. In a clustered web server environment with two WSAs, it is recommended that the first WSA points to the first AccessMatrix server and the second WSA points to the second AccessMatrix server to achieve a certain degree of load balancing and stickiness.

It is recommended to use source IP stickiness if the standard web load balancer is used. The web load balancer can be configured to poll a special JSP at *http://am_server1/am5/monitorme.jsp* to monitor the availability and response time of the AccessMatrix server.

To enable WSA to use the web load balancer, configure host1 URL to use the web load balancer virtual DNS or IP address.

On the other hand, to enable WSA to use its built-in software failover, perform the following steps:

1. Ensure that the same version of the **am.jar** is installed on both the WSA machine and AccessMatrix server.
2. On the client machines where WSA is installed, open the WSA configuration file, e.g. <*WSARP_PATH*>*/conf/wsa.conf*, to configure the load balancing and automatic failover features.
3. Edit this configuration file by making the following changes:

```
Code
```

```
<wsa id="usoagent" registered="1">
   <server>
      <authed>
         <host name="SG"
url="http://amserver1:8443/am5/xmlrpc"/>
         <host name="SG"
url="http://amserver2:8443/am5/xmlrpc"/>
         <connect timeout="30"/>
      </authed>
   </server>
…
</wsa>
```

> **Note**: Take note of the host names where the user should replace the values with the actual URLs of both first and second AccessMatrix servers respectively.

4. Save the changes made and restart the web application.

Capacity Planning

This section describes the specific steps in capacity planning when the number of WSAs grows over time.

The vertical scalability of the AccessMatrix server depends on a few factors:

- The type of container, e.g. Tomcat.
- The type of connectors, e.g. blocking or non-blocking network I/O of the container.
- How the HTTP(s) connections are kept alive.
- Use of queuing besides thread pool on the container.

Using today's multi-core architecture with WSA session cache that reduces remote calls to AccessMatrix servers, starting with two AccessMatrix servers should be sufficient to handle a throughput of 300-500 access requests per second.

However, if more WSAs are added with expected high load, capacity planning should be put into consideration. It can be started by calculating the maximum number of concurrent requests intended to be supported in all web applications. Ensure that the maximum number of concurrent requests are distributed to the number of AccessMatrix servers there is by using appropriate load balancing mechanisms as well as ensuring sufficient memory is allocated to each AccessMatrix server process.

If a high load is expected, configure AccessMatrix server container to use the non-blocking network I/O, e.g., Tomcat APR connector. The number of keep-alive connections allowed must not be significantly below the maximum concurrent requests to avoid deterioration of response time with peak load scenarios.

Configure JVM Settings for WSA

Normally, the recommended heap size for WSA is about 256MB. However, if the web server is running under a heavy load, it is required to change the default JVM parameter setting in the configuration file.

To configure:

1. Navigate to *<WSARP_PATH>/conf.d* and open **wsarp.conf**.
2. Modify the JVM parameter as shown below:

```
Code

  <IfModule wsa_module>
  ...
  JvmParams "-Xms128m -Xmx256m -
XX:+HeapDumpOnOutOfMemoryError"
  ...
</IfModule>
```

   - -XX:+HeapDumpOnOutOfMemoryError - This tells the JVM to generate a heap dump when an allocation from the Java heap or the permanent generation cannot be satisfied.
3. Save the changes.

Refer to the *AccessMatrix Platform Stress Test & Performance Tuning Guide* to know details on performance tuning and stress testing. You may contact i-Sprint Professional Services team to perform the assessment.

Support X-Forwarder-For and Forwarded Headers

UAM WSA RP instances may be deployed behind an HTTP load balancer. In order for UAM WSA RP to get the client's IP address, some of the load balancers can be configured to provide the client IP in an HTTP header. UAM WSA RP supports the **X-Forwarder-For** and **Forwarded** request headers to retrieve the client's IP address.

- The **X-Forwarded-For (XFF)** header is a de-facto standard header used to determine the originating IP address of a client connecting to a web server through an HTTP proxy or a load balancer.
- The **Forwarded** header is the standardized version of the X-Forwarded-For header, which contains information from the client-facing side of proxy servers that is altered or lost when a proxy is involved in the path of the request.

Enable Multiple SSO Proxy Support

This section describes the specific steps in configuring WSA Reverse Proxy (RP) to support multiple Single Sign-On (SSO) proxy.

Follow the steps below to enable this feature:

1. On the WSA RP server, open the relevant WSA configuration file, e.g. *<WSARP_PATH>\conf\wsa.conf* to configure the multiple SSO proxy.
2. Edit the WSA configuration file by making the following changes:

Code

```
<wsa id="usoagent">
   <server>
      <authed>
         <host name="SG"
url="http://sg.wsa.com:8080/am5/xmlrpc"/>
         <host name="MY"
url="http://my1.wsa.com:8080/am5/xmlrpc"/>
         <host name="MY"
url="http://my2.wsa.com:8080/am5/xmlrpc"/>
         <host name="CN"
url="http://cn.wsa.com:8080/am5/xmlrpc"/>
         <connect timeout="30"/>
      </authed>
   </server>
…
</wsa>
```

Notes:
- The URLs should be replaced with the actual URL values of the AccessMatrix UAM servers.
- The name can be any name to indicate the group of AccessMatrix UAM servers.

- AccessMatrix UAM servers with the same host name means these servers will be used for failover purpose.

3. Save the changes made and restart the web server.

For multiple SSO proxies to work, it is recommended to set the WSASRV cookie. This cookie will set the host name of each AccessMatrix UAM server.

For example, WSASRV=SG will connect to the "SG" AccessMatrix UAM server:

Code

```
<script>
function setHostname(name)
{
    document.cookie = 'WSASRV=' + name +
';domain=.wsa.com;path=/';

document.location='http://server.wsa.com/wsatest/private/'
}
</script>
<a href="javascript:setHostname('SG')">Singapore</a>
<a href="javascript:setHostname('MY')">Malaysia</a>
<a href="javascript:setHostname('CN')">China</a>
```

Note: If WSASRV cookie is not specified, the first host name specified in WSA configuration file (*<WSARP_PATH>\conf\wsa.conf*) will become the default cookie value.

Configure WSA RP to Load Apache MPM Worker

WSA Reverse Proxy only supports Apache that is configured to use Multi-Processing Module (MPM) worker. MPM implements a hybrid multi-process, multi-threaded server. Each child process will load its own WSA JVM into memory which will take up about 256MB of memory. Since each WSA RP will establish HTTP connections to the AccessMatrix server, you must also ensure that the potential total number of connections will not exceed the AccessMatrix server's capacity.

WSA Reverse Proxy has authorization cache stored in memory when running. If multiple MPM processes (Servers) are configured, "ServerLimit" is more than one, each process will have its own authorization cache and maintained and used within the child process itself (not shared). Thus, when configuring MPM with multiple processes, make sure that the amount of system memory is adequate.

The MPM worker configuration within the Apache HTTP server's startup elements needs to be configured correctly as described below. For a start, you should set the ServerLimit to 16 and StartServers to 2. Refer to Apache MPM worker for the detailed description of the entire configuration.

To enable the Apache server to load the WSA module:

1. Navigate to *<WSARP_PATH>/conf.d/* and open **wsarp.conf**.
2. Edit the configuration file by making changes to the following attributes:

```
Code

...

<IfModule mpm_worker_module>
    ServerLimit          16
    StartServers         2
    MaxRequestWorkers    150
    MinSpareThreads      25
    MaxSpareThreads      75
    ThreadsPerChild      25
</IfModule>
```

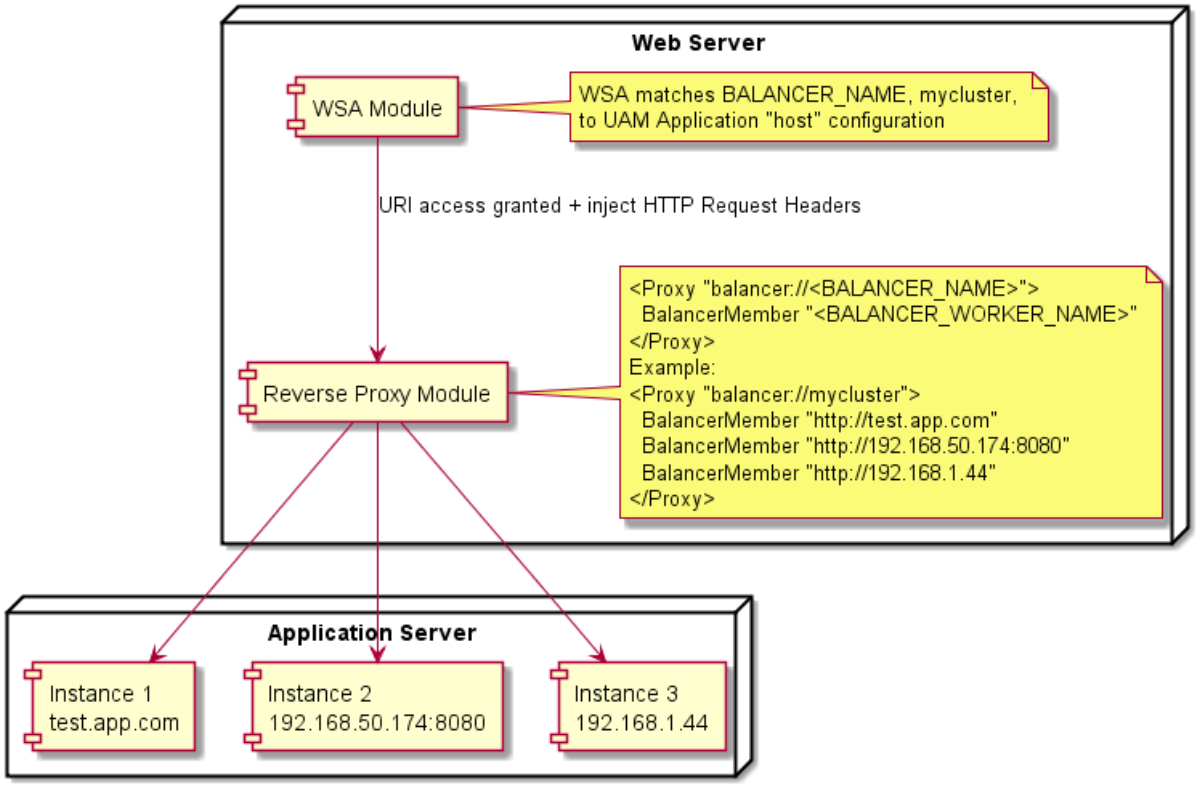
   - ServerLimit - Sets the maximum limit on configurable number of processes.
   - StartServers - Sets the number of child server processes created on startup.
   - MaxRequestWorkers - Sets the maximum number of connections that will be processes simultaneously.
   - MinSpareThreads - Minimum number of idle threads available to handle request spikes.
   - MaxSpareThreads - Maximum number of idle threads.
   - ThreadsPerChild - Sets the number of threads created by each child process.
3. Save the changes.

Support Load Balancing Using WSA Reverse Proxy

This section provides the steps to enable the Balancer Manager support in Apache HTTP Server using Web Security Agent (WSA) Reverse Proxy application.

Load balancers are used to effectively distribute incoming traffic across the application servers in a pool of servers to handle higher load and increase reliability.

UAM WSA Reverse Proxy Load Balancer Component Diagram



Configure the WSA Reverse Proxy Load Balancer

1. On the client machines where WSA RP is deployed, open the WSA configuration file, i.e. *<WSARP_PATH>/conf.d/wsarp.conf*, to configure the load balancing.
2. Add the following proxy configuration inside the <VirtualHost> tag:

Code

```
<VirtualHost *:80>
    ...
    <Proxy "balancer://<BALANCER_NAME>">
        BalancerMember "<BALANCER_WORKER_NAME>"
    </Proxy>
    <LocationMatch "/wsatest/">
    ...
</VirtualHost>
```

   - BALANCER_NAME - This is the identifier name of the load balancer defined and assigned to group the multiple instances of the target applications, e.g. balancer://mycluster. The balancer://mycluster identifier is only an identifier; this can be any value as long as you use the balancer:// prefix. This name will be used as the **Host** value in Load Balancer Target Application.

- BALANCER_WORKER_NAME - This refers to the URL of the target application(s) that shares the load of the incoming requests, e.g. https://hostA:80. This is the backend server, which is also known as a BalanceMember.

3. Modify the configuration inside <LocationMatch> tag and save the changes:

```
Code

<LocationMatch "<CONTEXT_PATH>">
    ProxyPass
"balancer://<BALANCER_NAME>/<CONTEXT_PATH>/"
    ProxyPassReverse
"balancer://<BALANCER_NAME>/<CONTEXT_PATH>/"
</LocationMatch>
```

- CONTEXT_PATH - This is the context path of the Web Security Agent (WSA) application, e.g. /wsatest. You can add different sets of <LocationMatch> for different WSA applications, just ensure that it's using the same balancer name.

> **Note**: The BALANCER_NAME in this tag should be the same with the balancer name defined in the <Proxy> tag.

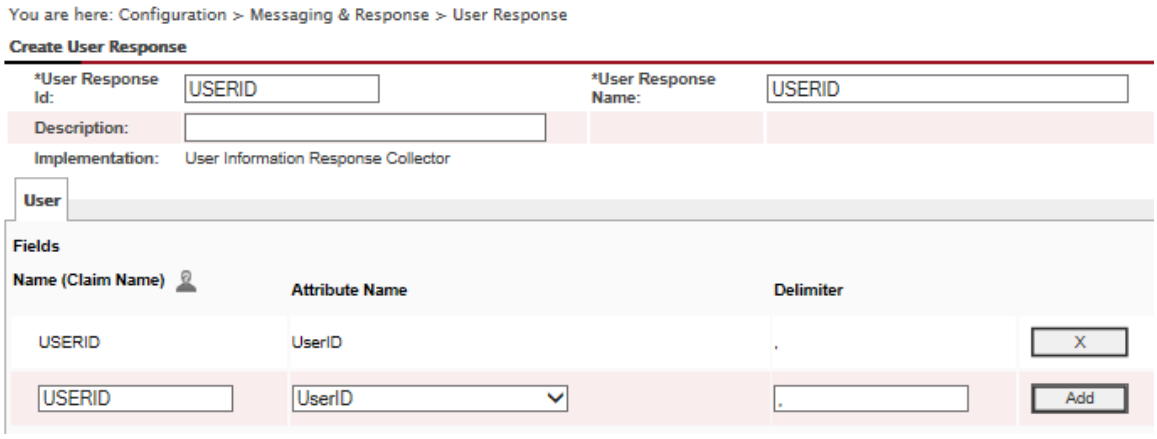
Below is an example of the **wsarp.conf** configurations detailed in this section. It shows that a balancer set is created, with the name "mycluster". It includes 3 backend servers, which httpd calls BalancerMembers. Any requests for the target application (e.g. /wsatest) will be proxied to one of the 3 backends.

```
Code

<VirtualHost server.wsa.com:80>
    ServerName server.wsa.com
    ...
    <Proxy "balancer://mycluster">
        BalancerMember "http://test.app.com"
        BalancerMember "http://192.168.50.174:8080"
        BalancerMember "http://192.168.1.44"
    </Proxy>
    <LocationMatch "/wsatest">
        ProxyPass "balancer://mycluster/wsatest/"
        ProxyPassReverse
"balancer://mycluster/wsatest/"
    </LocationMatch>
</VirtualHost>
```

Create the Load Balancer Target Application

1. Before creating the target application, configure the User Response to return certain user-specific information (e.g. USERID). Navigate to the **User Response Page** by selecting **Administration Dashboard > Configuration > Messaging & Response > User Response**.
2. Create a user information response collector (e.g "USERID") to map the HTTP header to the user attribute. These HTTP headers used for the mapping can be found in the **index.jsp**.

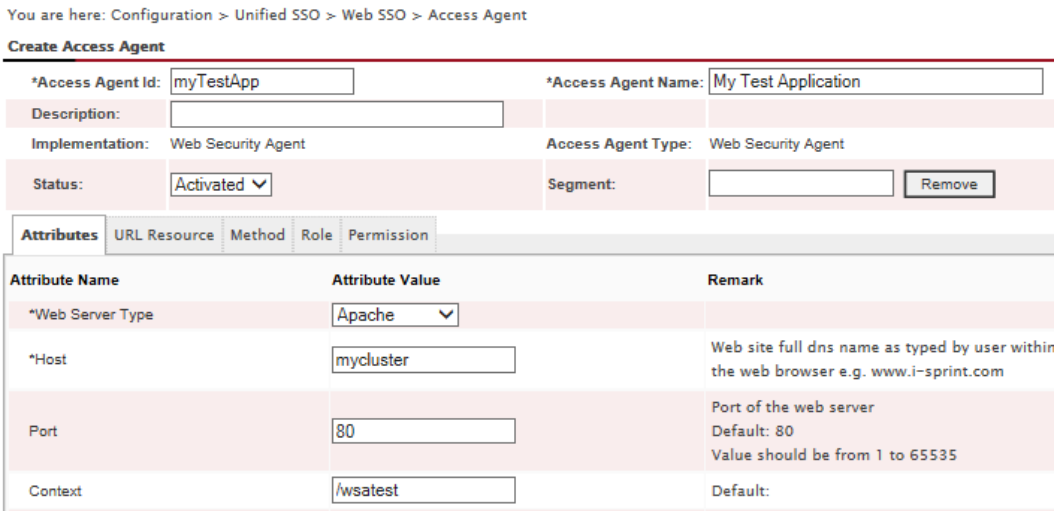


3. Save the changes.
4. Navigate to the **Access Agent Page** by selecting **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.
5. Create a Web Security Agent called "myTestApp" for the target web application context "wsatest" with the following information, and prefix all the URL fields with the same context as shown in following figure:
    a. **Host**: "mycluster"

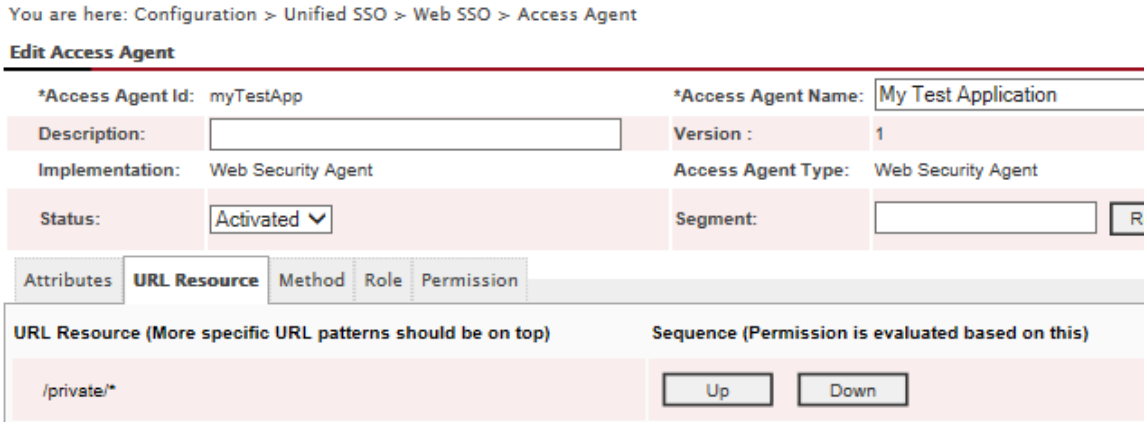
    > **Note**: The **Host** value should be the BALANCER_NAME defined in **wsarp.conf**.

    b. **Port**: "80"

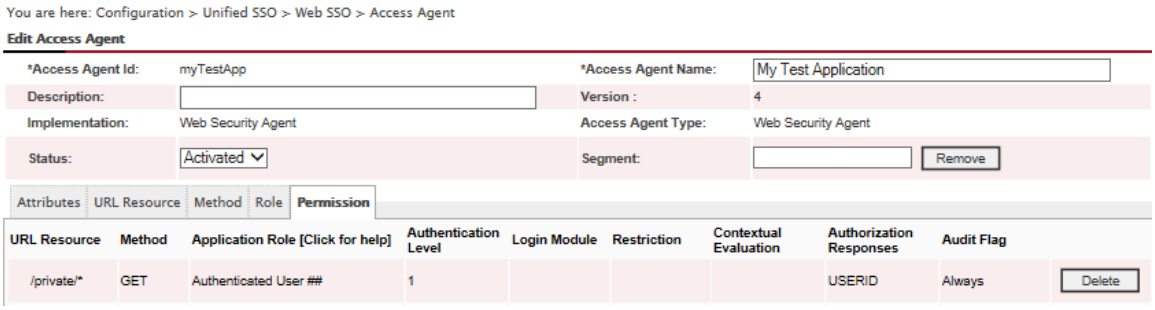
c. **Context**: "/wsatest"



6. Click on the **URL Resource** tab to create the following URL:



7. Click on the **Permission** tab to create the following permissions:



With the above permission configuration, WSA Reverse Proxy will inject a response configuration called "USERID" into HTTP request headers at "/private/*" URL.

8. Save the changes.

Enable Balancer Manager Support (Optional)

Balancer manager enables dynamic update of balancer members. This will display the current working configuration and status of the enabled balancers and workers currently in use. It also allows for dynamic, runtime, on-the-fly reconfiguration of all balancers, including adding new BalancerMembers (workers) to an existing balancer.

Perform the following steps to enable balancer-manager support:

1. On the client machines where WSA RP is deployed, open the relevant WSA configuration file, i.e. *<WSARP_PATH>/conf.d/wsarp.conf*.
2. Add the following configuration to enable the embedded balancer-manager application:

Code

```
<Location "<LB_CONTEXT_PATH>">
    SetHandler balancer-manager
    Require <ACCESS_TYPE>
</Location>
```

   - LB_CONTEXT_PATH - This is the context path of the Web Security Agent (WSA) application that will access the Load Balancer Manager, e.g. /balancer. The value should be the same with the "context" value defined in WSA application.
   - ACCESS_TYPE - This is the access type of the Require directive that provides different ways to allow or deny access to resources. Access can be controled by hostname, IP Address or IP Address range.
     - Require ip - The ip provider allows access to the server to be controlled based on the IP address of the remote client. When Require ip ip-address is specified, then the request is allowed access if the IP address matches. This can be a full IP Address, partial IP Address, or network/netmask pair.
     - Require host - The host provider allows access to the server to be controlled based on the host name of the remote client. When Require host host-name is specified, then the request is allowed access if the host name matches.
     - Require forward-dns - The forward-dns provider allows access to the server to be controlled based on simple host names. When Require forward-dns host-name is specified, all IP addresses corresponding to host-name are allowed access.

- Require local - The local provider allows access to the server if any of the following conditions is true:
    - the client address matches 127.0.0.0/8
    - the client address is ::1
    - both the client and the server address of the connection are the same

> **Note**: Refer to https://httpd.apache.org/docs/2.4/mod/mod_authz_host.html#requiredirectives for more details.

Below is a sample **wsarp.conf** configuration with balance-manager support:

Code

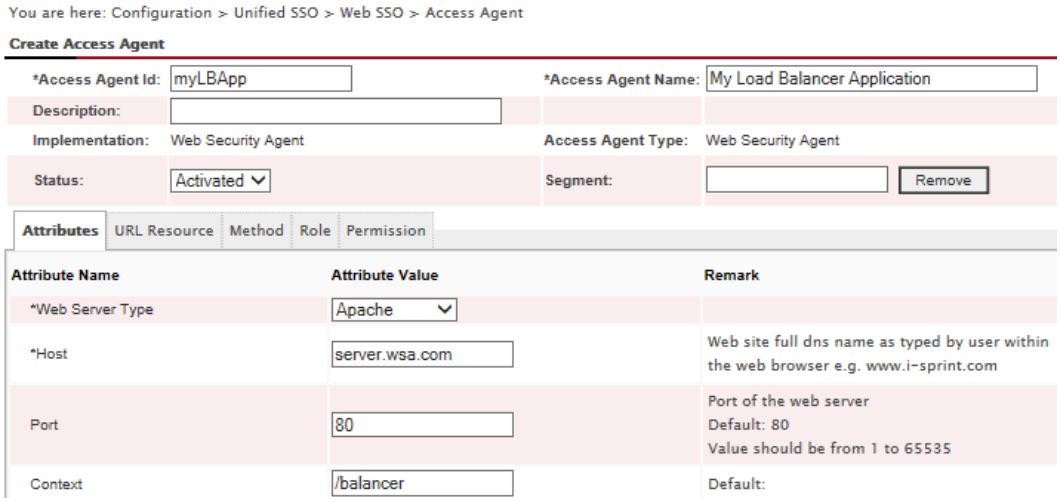
```
<VirtualHost server.wsa.com:80>
    ServerName server.wsa.com
    LogLevel error
    AllowEncodedSlashes On
    ProxyRequests Off

    <Proxy "balancer://mycluster">
        BalancerMember "http://test.app.com"
        BalancerMember "http://192.168.50.174:8080"
        BalancerMember "http://192.168.1.44"
    </Proxy>
    <LocationMatch "/wsatest">
        ProxyPass "balancer://mycluster/wsatest/"
        ProxyPassReverse
"balancer://mycluster/wsatest/"
    </LocationMatch>
</VirtualHost>
<Location "/balancer">
    SetHandler balancer-manager
    Require host example.com
</Location>
```

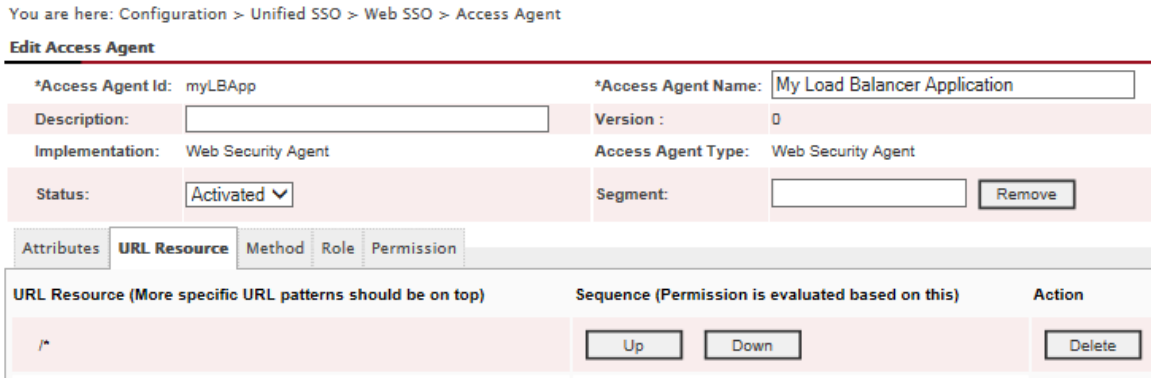
Create WSA Application to Access the Load Balancer Manager

1. Navigate to the **Access Agent Page** by selecting **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.
2. Create a Web Security Agent called "myLBApp" for the target web application context "balancer" with the following information, and prefix all the URL fields with the same context as shown in following figure:
    - **Host**: "server.wsa.com"

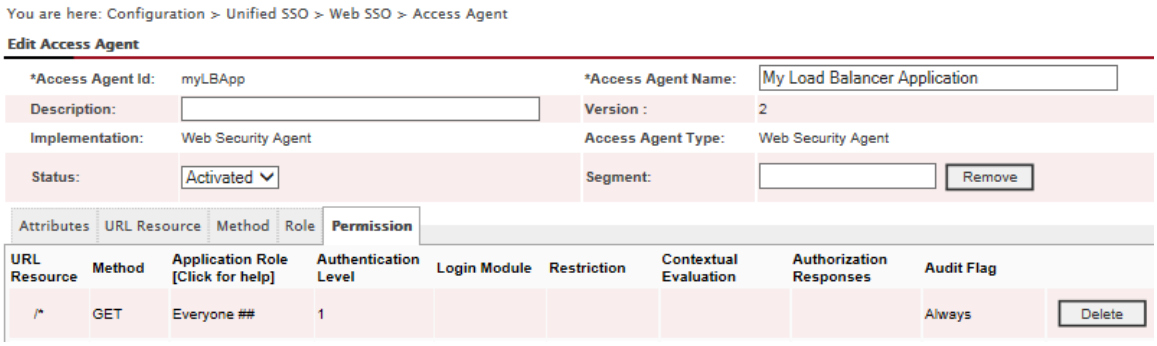
- **Port**: "80"
- **Context**: "/balancer"



3. Click on the **URL Resource** tab to create the following URL:



4. Click on the **Permission** tab to create the following permissions:



5. Save the changes.

Configure SAML SSO to WSA RP

WSA Reverse Proxy supports SAML assertion (response) to make SAML deployment easier for Service Provider. WSA acts as the SAML ACS (Assertion Consumer Service) endpoint to allow SAML SSO from an

external SAML Identity Provider using IdP-initiated SAML Browser SSO Profile. WSA RP will read the SAMLResponse and RelayState POST parameters (HTTP-POST binding). Then, WSA RP calls login API with the SAML assertion. After successful validation of the SAML assertion. Authentication realm with SAML Web Browser SSO Lookup and Login modules need to be used. The SAML Realm ID is configured as below in the configuration file (*<WSARP_PATH>/conf.d/wsa.conf*):

Code

```
<wsa>
  ...
    <SAML realmId="WSASAML"/>
  ...
</wsa>
```

WSA SAML ACS endpoint can be accessed on this path */wsa/wsasaml*.

WSA RP also supports multiple realms to support a scenario where WSA acts as the ACS URL for multiple SAML Service Providers or multiple SAML Identity Providers. The multiple SAML Realm IDs can be configured as below:

Code

```
<wsa>
  ...
    <SAML realmId="WSASAML" allowedRealmIds="<realmA>,
<realmB>, <...>"/>
  ...
</wsa>
```

In this configuration, the default realm Id (e.g. "WSASAML") is defined in realmId. Additional realm Ids can be defined in allowedRealmIds, and only the specified realm Ids are allowed.

The ACS URL for the default SAML realm is */wsa/wsasaml*, whereas the other mapped SAML realms can be accessed using */wsa/wsasaml/<realmId>*, where the realmId is listed in the allowedRealmIds configuration.

Configure CLP URL

WSA RP may use the Common Login Page (CLP) to handle the user authentication. To support this, it is required to configure the CLP URL. Refer to *AccessMatrix Common Login Page Deployment Guide - Configure CLP for WSA in WSA Reverse Proxy Component* for the configuration steps.

Configure Access Log Format

WSA RP allows the printing of custom user information in the Apache access log for various authentication scenarios. This section explains the configurations needed to configure the printing of the desired user attribute in the following scenarios, as described in the table:

Scenario	Description
Login Succeeded	User successfully logs into application.
Change Password	User initiates password change for application.
Change Password Succeeded	User successfully changes password.
Reauthentication	User is prompted for reauthentication.

Refer to the User Fields Attribute Name sub-section of *Common Administration Guide - Manage Messaging & Response* section for a detailed list of the attributes that can be printed in the access log.

> **Note**: You must first deploy your WSA Reverse Proxy web application before performing the following steps. Refer to Deploy Sample Application of WSA Reverse Proxy for the instructions.

The following steps demonstrate how to configure the printing of custom user information in the access log (using the "UserID" attribute as an example):

Create User Response with Custom Attribute

1. Navigate to **Administration Dashboard > Configuration > Messaging & Response > User Response** in the Admin Console.
2. Click on the **Create** button and select "User Information Response Collector" from the **Implementation** drop-down list.
3. Fill in the ***User Response Id** and **User Response Name** fields.
4. Select the **Attribute Name** to be printed in the access log (e.g. "UserID") and enter a corresponding **Claim Name** (e.g. "USERID").
5. Click on the **Add** button.
6. Click on the **Save** button.

Add User Response to Authorization Responses in Access Agent

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.
2. Search for the relevant Web Security Agent for your application and click on it.
3. Click on **Edit** and navigate to the **Permission** tab.
4. Set the necessary configurations for:
    a. **URL Resource** - Select "/private/*".
    b. **Method** - Select "GET".
    c. **Application Role*** - Select "Authenticated User".
    d. **Authentication Level** - Select the authentication level corresponding to the relevant login module(s) (e.g. "1").
    e. **Audit Flag** - Select your desired audit flag to choose when the log should be updated (e.g. "Always").
    f. **Login Module** - Select the relevant login module(s) as appropriate.
5. Under **Authorization Responses**, click on the **Modify** button and tick the checkbox of the newly-created user response. Click **OK** to close the dialog box.
6. Click **Add**.
7. Repeat steps 4-6, but select "POST" for **Method** this time.
8. Click on the **Save** button.

Update Apache HTTP Server Configuration File

1. Open the Apache HTTP server configuration file located at *<WSARP_PATH>/conf/httpd.conf*.
2. Search for the IfModule log_config_module keywords and add %<claim name>i to the LogFormat line accordingly. Using the "USERID" claim name as an example:

Code

```
<IfModule log_config_module>
#
# The following directives define some format nicknames
for use with
# a CustomLog directive (see below).
#
LogFormat "%h %l %{USERID}i %t \"%r\" %>s %b
\"%{Referer}i\" \"%{User-Agent}i\"" combined
LogFormat "%h %l %u %t \"%r\" %>s %b" common
```

```
#LogFormat "%{Referer}i -> %U" referer
#LogFormat "%{User-agent}i" agent
```

3. Save your changes and close the file.
4. Perform a test login as necessary. Based on the example in the previous steps, the log should now include the "UserID" attribute. For example:

```
192.168.1.44- demouser - [25/May/2022:09:19:18+0800] "GET /app1/private/ HTTP/1.1"200 1503
```

4.2.2. WSA RP (Native) Advanced Configurations

Advanced features are available to cater for individual organizations' needs. It is generally not required to configure these features.

Enable WSA RP Default Authentication Pages (Optional)

WSA RP provides default authentication pages to support user authentication feature. By default, this setting is disabled for hardening purposes.

To enable WSA RP's default authentication pages:

1. Navigate to *<WSARP_PATH>/wsa/conf/* and open **wsarp.conf**.
2. Edit the configuration file by making the following changes:

Code

```
Define wsaAuthRoot /etc/httpd/wsa/auth
  ...
  <VirtualHost *:443>
      ...
      DocumentRoot ${wsaAuthRoot}
      <Directory ${wsaAuthRoot}>
           Require all granted
      </Directory>
      ...
  </VirtualHost>
```

Configure High Availability

WSA supports the following high availability option:

- Standard web load balancer (recommended if there are more than 2 AccessMatrix servers)

In a clustered web server environment with two WSAs, it is recommended that the first WSA points to the first AccessMatrix server and the second WSA points to the second AccessMatrix server to achieve a certain degree of load balancing and stickiness.

It is recommended to use source IP stickiness if the standard web load balancer is used. The web load balancer can be configured to poll a special JSP at *http://am_server1/am5/monitorme.jsp* to monitor the availability and response time of the AccessMatrix server.

To enable WSA to use the web load balancer, configure host1 URL to use the web load balancer virtual DNS or IP address.

Capacity Planning

This section describes the specific steps in capacity planning when the number of WSAs grows over time.

The vertical scalability of the AccessMatrix server depends on a few factors:

- The type of container, e.g. Tomcat.
- The type of connectors, e.g. blocking or non-blocking network I/O of the container.
- How the HTTP(s) connections are kept alive.
- Use of queuing besides thread pool on the container.

Using today's multi-core architecture with WSA session cache that reduces remote calls to AccessMatrix servers, starting with two AccessMatrix servers should be sufficient to handle a throughput of 300-500 access requests per second.

However, if more WSAs are added with expected high load, capacity planning should be put into consideration. It can be started by calculating the maximum number of concurrent requests intended to be supported in all web applications. Ensure that the maximum number of concurrent requests are distributed to the number of AccessMatrix servers there is by using appropriate load balancing mechanisms as well as ensuring sufficient memory is allocated to each AccessMatrix server process.

If a high load is expected, configure AccessMatrix server container to use the non-blocking network I/O, e.g., Tomcat APR connector. The number of keep-alive connections allowed must not be significantly below the maximum concurrent requests to avoid deterioration of response time with peak load scenarios.

Support X-Forwarder-For and Forwarded Headers

UAM WSA RP instances may be deployed behind an HTTP load balancer. In order for UAM WSA RP to get the client's IP address, some of the load balancers can be configured to provide the client IP in an HTTP header. UAM WSA RP supports **X-Forwarder-For** and **Forwarded** request headers to retrieve the client's IP address.

- The **X-Forwarded-For (XFF)** header is a de-facto standard header used to determine the originating IP address of a client connecting to a web server through an HTTP proxy or a load balancer.
- The **Forwarded** header is the standardised version of the X-Forwarded-For header, which contains information from the client-facing side of proxy servers that is altered or lost when a proxy is involved in the path of the request.

Configure WSA RP to Load Apache MPM Worker

WSA Reverse Proxy only supports Apache that is configured to use Multi-Processing Module (MPM) worker. MPM implements a hybrid multi-process, multi-threaded server. Since each WSA RP will establish HTTP connections to the AccessMatrix server, you must also ensure that the potential total number of connections will not exceed the AccessMatrix server's capacity.

WSA Reverse Proxy has authorization cache stored in memory when running. If multiple MPM processes (Servers) are configured, "ServerLimit" is more than one, each process will have its own authorization cache and maintained and used within the child process itself (not shared). Thus, when configuring MPM with multiple processes, make sure that the amount of system memory is adequate.

The MPM worker configuration within the Apache HTTP server's startup elements needs to be configured correctly as described below. For a start, you should set the ServerLimit to "16" and StartServers to "2". Refer to Apache MPM worker for the detailed description of the entire configuration.

To enable Apache server to load the WSA module:

1. Navigate to *<WSARP_PATH>/wsa/conf/* and open **wsarp.conf**.
2. Edit the configuration file by making changes to the following properties:

Code

```
...

<IfModule mpm_worker_module>
    ServerLimit          1
    StartServers         1
    MaxRequestWorkers    50
    MinSpareThreads      25
    MaxSpareThreads      40
    ThreadsPerChild      50
</IfModule>
```

   - ServerLimit - Sets the maximum limit on configurable number of processes.

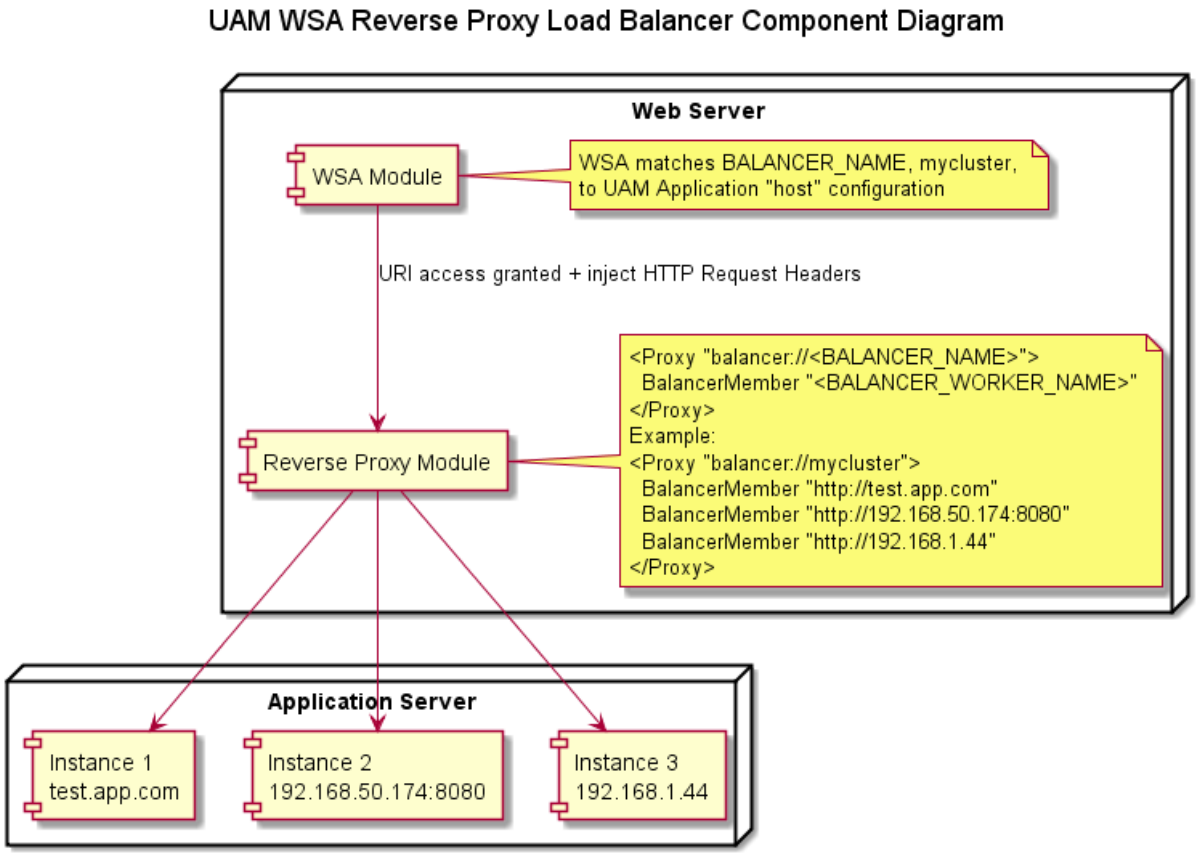
- StartServers - Sets the number of child server processes created on startup.
- MaxRequestWorkers - Sets the maximum number of connections that will be processes simultaneously.
- MinSpareThreads - Minimum number of idle threads available to handle request spikes.
- MaxSpareThreads - Maximum number of idle threads.
- ThreadsPerChild - Sets the number of threads created by each child process.

3. Save the changes.

Support Load Balancing Using WSA Reverse Proxy

This section provides the steps to enable the Balancer Manager support in Apache HTTP Server using Web Security Agent (WSA) Reverse Proxy application.

Load balancers are used to effectively distribute incoming traffic across the application servers in a pool of servers to handle higher load and increase reliability.



Configure the WSA Reverse Proxy Load Balancer

1. On the client machines where WSA RP is deployed, open the WSA configuration file, i.e. *<WSARP_PATH>/wsa/conf/wsarp.conf* to configure the load balancing.
2. Add the following proxy configuration inside the <VirtualHost> tag:

```
Code

<VirtualHost *:80>
    ...
    </Directory>
    <Proxy "balancer://<BALANCER_NAME>">
        BalancerMember "<BALANCER_WORKER_NAME>"
    </Proxy>
    <LocationMatch "/wsatest/">
    ...
</VirtualHost>
```

   - BALANCER_NAME - This is the identifier name of the load balancer defined and assigned to group the multiple instances of the target applications, e.g. balancer://mycluster. The balancer://mycluster identifier is only an identifier; this can be any value as long as you use the balancer:// prefix. This name will be used as **Host** value in Load Balancer Target Application.
   - BALANCER_WORKER_NAME - This refers to the URL of the target application(s) that shares the load of the incoming requests, e.g. https://hostA:80. This is the backend server, which is also known as BalanceMember.

3. Modify the configuration inside the <LocationMatch> tag and save the changes:

```
Code

<LocationMatch "<CONTEXT_PATH>">
    ProxyPass
"balancer://<BALANCER_NAME>/<CONTEXT_PATH>/"
    ProxyPassReverse
"balancer://<BALANCER_NAME>/<CONTEXT_PATH>/"
</LocationMatch>
```

   - CONTEXT_PATH - This is the context path of the Web Security Agent (WSA) application, e.g. /wsatest. You can add different sets of <LocationMatch> for different WSA applications, just ensure that it's using the same balancer name.

     Note: The BALANCER_NAME in this tag should be the same with the balancer name defined in the <Proxy> tag.

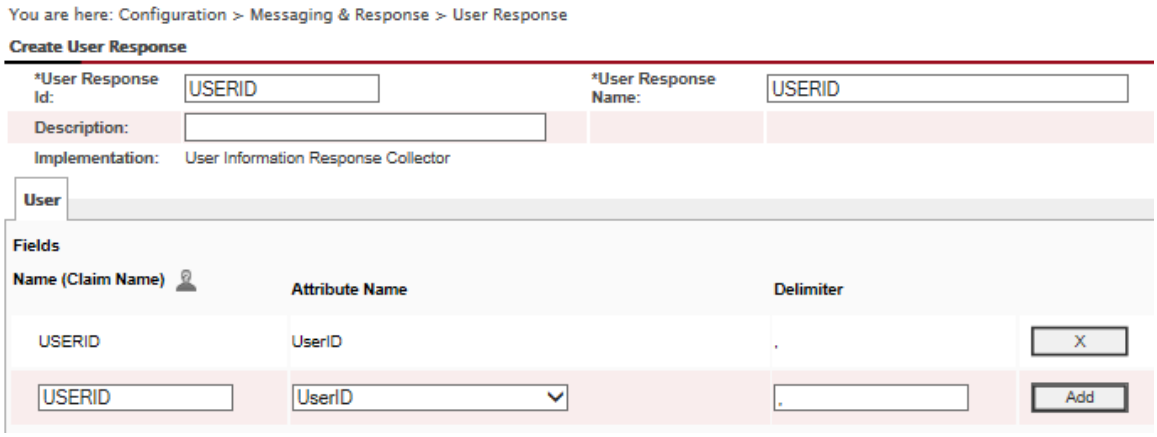
The table below is an example of **wsarp.conf** configuration. It shows that a balancer set is created, with the name "mycluster". It includes 3 backend servers, which httpd calls BalancerMembers. Any requests for the target application (e.g. /wsatest) will be proxied to one of the 3 backends.

Code

```
<VirtualHost *:80>
    ServerName ${wsaServerName}
    ...
    <Proxy "balancer://mycluster">
        BalancerMember "http://test.app.com"
        BalancerMember "http://192.168.50.174:8080"
        BalancerMember "http://192.168.1.44"
    </Proxy>
    <LocationMatch "/wsatest">
        ProxyPass "balancer://mycluster/wsatest/"
        ProxyPassReverse
"balancer://mycluster/wsatest/"
    </LocationMatch>
</VirtualHost>
```

Create the Load Balancer Target Application

1. Before creating the target application, configure the user response to return certain user-specific information (e.g. "USERID"). Navigate to the **User Response Page** by selecting **Administration Dashboard > Configuration > Messaging & Response > User Response**.
2. Create a User Information Response Collector (e.g "USERID") to map the HTTP header to the user attribute. These HTTP headers used for the mapping can be found in **index.jsp**.



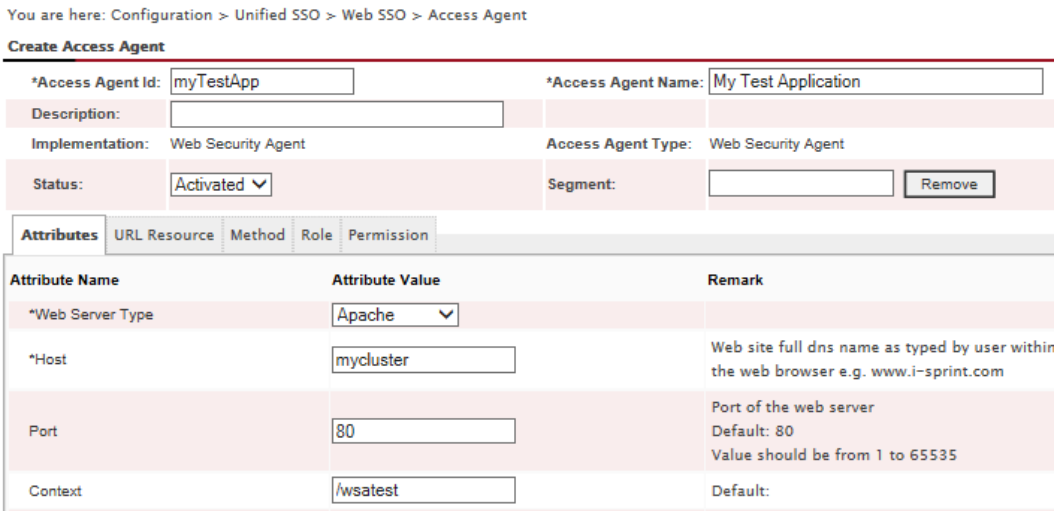
3. Save the changes.
4. Navigate to the **Access Agent Page** by selecting **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.

5. Create a Web Security Agent called "myTestApp" for target web application context "wsatest" with the following information, and prefix all the URL fields with the same context as shown in following figure:

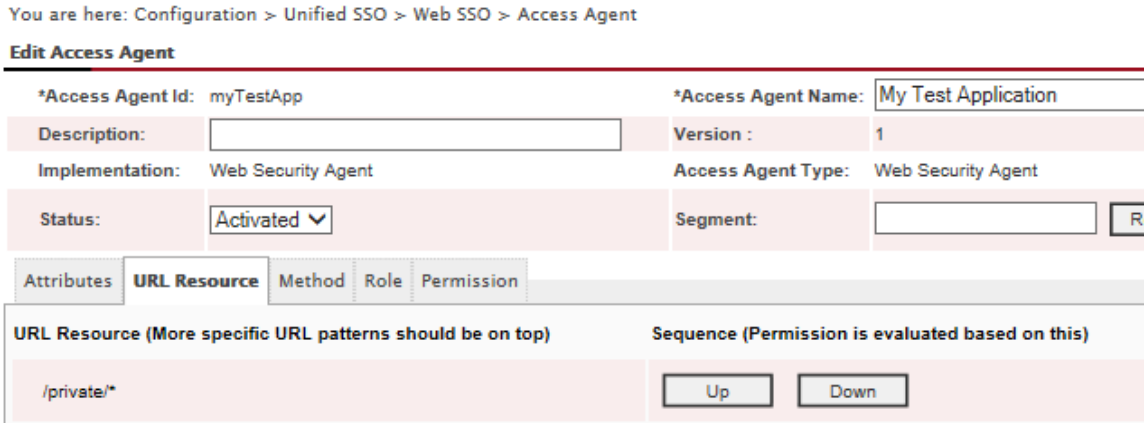
   - **Host**: "mycluster"

     > **Note**: The **Host** value should be the BALANCER_NAME defined in **wsarp.conf**.

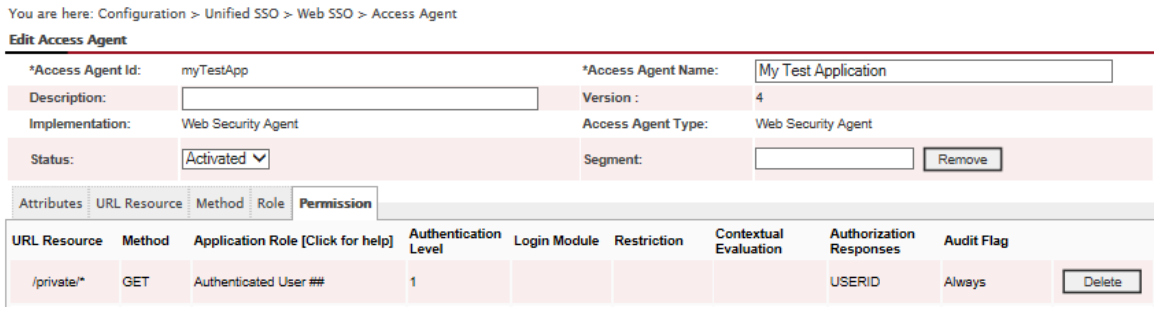
   - **Port**: "80"
   - **Context**: "/wsatest"



6. Click on the **URL Resource** tab to create the following URL:



7. Click on the **Permission** tab to create the following permissions:



   With the above permission configuration, WSA Reverse Proxy will inject a response configuration called "USERID" into HTTP request headers at the "/private/*" URL.

8. Save the changes.

Enable Balancer Manager Support (Optional)

Balancer manager enables dynamic update of balancer members. This will display the current working configuration and status of the enabled balancers and workers currently in use. It also allows for dynamic, runtime, on-the-fly reconfiguration of all balancers, including adding new BalancerMembers (workers) to an existing balancer.

Perform the following steps to enable balancer-manager support:

1. On the client machines where WSA RP is deployed, open the relevant WSA configuration file, i.e. *<WSARP_PATH>/wsa/conf/wsarp.conf*.
2. Add the following configuration to enable the embedded balancer-manager application:

   Code

   ```
   <Location "<LB_CONTEXT_PATH>">
       SetHandler balancer-manager
       Require <ACCESS_TYPE>
   </Location>
   ```

   - LB_CONTEXT_PATH - This is the context path of the Web Security Agent (WSA) application that will access the Load Balancer Manager, e.g. /balancer. The value should be the same with the **Context** value defined in WSA application.
   - ACCESS_TYPE - This is the access type of the Require directive that provides different ways to allow or deny access to resources. Access can be controled by hostname, IP Address or IP Address range.

- Require ip - The ip provider allows access to the server to be controlled based on the IP address of the remote client. When Require ip ip-address is specified, then the request is allowed access if the IP address matches. This can be a full IP Address, partial IP Address, or network/netmask pair.
- Require host - The host provider allows access to the server to be controlled based on the host name of the remote client. When Require host host-name is specified, then the request is allowed access if the host name matches.
- Require forward-dns - The forward-dns provider allows access to the server to be controlled based on simple host names. When Require forward-dns host-name is specified, all IP addresses corresponding to host-name are allowed access.
- Require local - The local provider allows access to the server if any of the following conditions is true:
    - the client address matches 127.0.0.0/8
    - the client address is ::1
    - both the client and the server address of the connection are the same

    Note: Refer to https://httpd.apache.org/docs/2.4/mod/mod_authz_host.html#requiredirectives for more details.

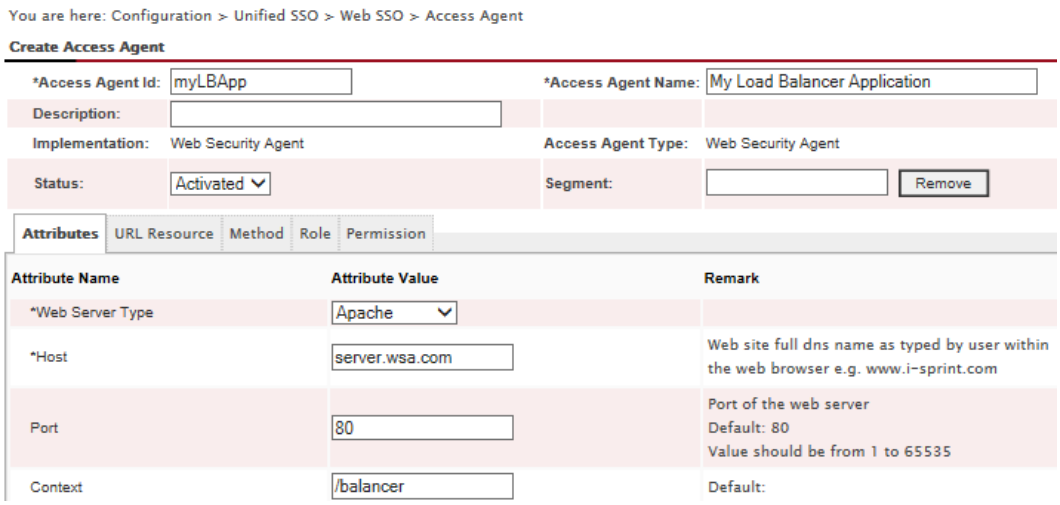
The table below is an example of **wsarp.conf** configuration with balance-manager support:

Code

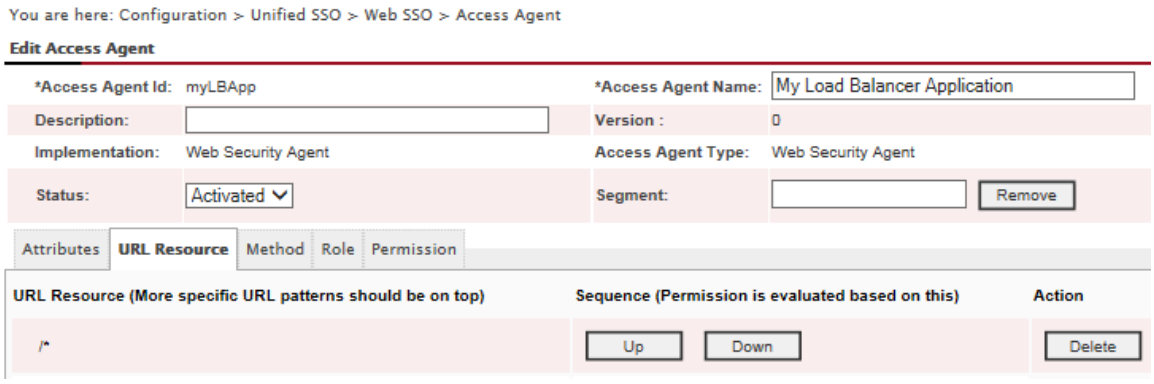
```
<VirtualHost *:80>
    ServerName ${wsaServerName}
    ...
    <Proxy "balancer://mycluster">
        BalancerMember "http://test.app.com"
        BalancerMember "http://192.168.50.174:8080"
        BalancerMember "http://192.168.1.44"
    </Proxy>
    <LocationMatch "/wsatest">
        ProxyPass "balancer://mycluster/wsatest/"
        ProxyPassReverse
"balancer://mycluster/wsatest/"
    </LocationMatch>
</VirtualHost>
<Location "/balancer">
    SetHandler balancer-manager
    Require host example.com
</Location>
```

Create WSA Application to Access the Load Balancer Manager

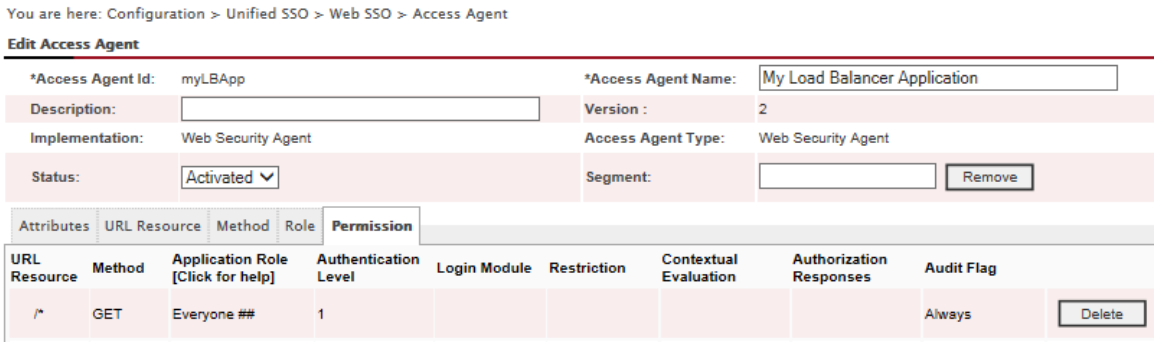
1. Navigate to **Access Agent Page** by selecting **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.
2. Create a Web Security Agent called "myLBApp" for target web application context "balancer" with the following information, and prefix all the URL fields with the same context as shown in following figure:
    - **Host**: "server.wsa.com"
    - **Port**: "80"
    - **Context**: "/balancer"



3. Click on the **URL Resource** tab to create the following URL:



4. Click on the **Permission** tab to create the following permissions:



5. Save the changes.

Configure SAML SSO to WSA RP

WSA Reverse Proxy supports SAML assertion (response) to make SAML deployment easier for Service Provider. WSA acts as the SAML ACS (Assertion Consumer Service) endpoint to allow SAML SSO from an external SAML Identity Provider using IdP-initiated SAML Browser SSO Profile. WSA RP will read the SAMLResponse and RelayState POST parameters (HTTP-POST binding). Then, WSA RP calls login API with the SAML assertion. After successful validation of the SAML assertion. Authentication realm with SAML Web Browser SSO lookup and login modules need to be used. The SAML realm Id is configured as below in the configuration file (*<WSARP_PATH>/wsa/conf/wsarp.conf*):

Code

```
<IfModule wsa_module>
    ...
    SamlRealmIds "SAMLRealm"
</IfModule>
```

WSA SAML ACS endpoint can be accessed on this path */wsa/wsasaml*.

WSA RP also supports multiple realms to support a scenario where WSA acts as the ACS URL for multiple SAML Service Providers or multiple SAML Identity Providers. The multiple SAML realm Ids can be configured as below:

Code

```
<IfModule wsa_module>
    ...
    SamlRealmIds "SAMLRealm,SAMLRealmA,SAMLRealmB"
</IfModule>
```

For multiple realms, the first SAML realm Id is considered the default SAML realm. It can be accessed via */wsa/wsasaml*. For the other SAML realm Ids, they can be accessed via */wsa/wsasaml/<realmId>*. Following the above examples, SAMLRealmA is accessed via */wsa/wsasaml/SAMLRealmA*, whereas SAMLRealmB is accessed via */wsa/wsasaml/SAMLRealmB*.

Configure CLP URL

WSA RP may use the Common Login Page (CLP) to handle the user authentication. To support this, it is required to configure the CLP URL. Refer to *AccessMatrix Common Login Page Deployment Guide - Configure CLP for WSA in WSA Reverse Proxy Component* for the configuration steps.

Configure Access Log Format

WSA RP allows the printing of custom user information in the Apache access log for various authentication scenarios. This section explains the configurations needed to configure the printing of the desired user attribute in the following scenarios, as described in the table:

Scenario	Description
Login Succeeded	User successfully logs into application.
Change Password	User initiates password change for application.
Change Password Succeeded	User successfully changes password.
Reauthentication	User is prompted for reauthentication.

Refer to the User Fields Attribute Name sub-section of *Common Administration Guide - Manage Messaging & Response* for a detailed list of the attributes that can be printed in the access log.

> **Note**: You must first deploy your WSA Reverse Proxy web application before performing the following steps. Refer to Deploy Sample Application of WSA Reverse Proxy for the instructions.

The following steps demonstrate how to configure the printing of custom user information in the access log (using the "UserID" attribute as an example):

Create User Response with Custom Attribute

1. Navigate to **Administration Dashboard > Configuration > Messaging & Response > User Response** in the Admin Console.
2. Click on the **Create** button and select "User Information Response Collector" from the **Implementation** drop-down list.
3. Fill in the ***User Response Id** and **User Response Name** fields.
4. Select the **Attribute Name** to be printed in the access log (e.g. "UserID") and enter a corresponding **Claim Name** (e.g. USERID).
5. Click on the **Add** button.
6. Click on the **Save** button.

Add User Response to Authorization Responses in Access Agent

1. Navigate to **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.
2. Search for the relevant Web Security Agent for your application and click on it.
3. Click on **Edit** and navigate to the **Permission** tab.
4. Set the necessary configurations for:
    a. **URL Resource** - Select "/private/*".
    b. **Method** - Select "GET".
    c. **Application Role*** - Select "Authenticated User".
    d. **Authentication Level** - Select the authentication level corresponding to the relevant login module(s) (e.g. "1").
    e. **Audit Flag** - Select your desired audit flag to choose when the log should be updated (e.g. "Always").
    f. **Login Module** - Select the relevant login module(s) as appropriate.
5. Under **Authorization Responses**, click on the **Modify** button and tick the checkbox of the newly-created user response. Click **OK** to close the dialog box.
6. Click **Add**.
7. Repeat steps 4-6, but select "POST" for **Method** this time.
8. Click on the **Save** button.

Update Apache HTTP Server Configuration File

1. Open the Apache HTTP server configuration file located at *<WSARP_PATH>/conf/httpd.conf*.
2. Search for the IfModule log_config_module keywords and add %<claim name>i to the LogFormat line accordingly. Using the "USERID" claim name as an example:
3. Save your changes and close the file.
4. Perform a test login as necessary. Based on the example in the previous steps, the log should now include the "UserID" attribute. For example:

```
192.168.1.44- demouser - [25/May/2022:09:19:18+0800] "GET /app1/private/ HTTP/1.1"200 1503
```

5. UAM WSA Reverse Proxy Sample Application

This section describes how to deploy the sample application of WSA Reverse Proxy.

Sample Demo - Tomcat Web Application

This section describes the step by step setup of a Tomcat demo web application with examples on using the Http Request Header for an application to obtain user information. For quick setup in training, it is possible to export the UAM application via the **CSV Data Export/Import** utility in the admin console.

Having applied and configured the WSA reverse proxy described in the previous sections, the Administrator may proceed to configure WSA Reverse Proxy to support for HTTP request headers, if applicable. This is done by means of user information response configuration via the Administration Console. Refer to *UAM Administration Guide* for the detailed instructions on how to administer application roles as well as to configure user information response and UAM applications generally.

The following pages demonstrate how to configure WSA Reverse Proxy to support for HTTP request headers as well as a test to verify the configuration:

> **Note**: This demonstration uses the Apache Tomcat web server bundled with the AccessMatrix Installer as both the AccessMatrix UAM Server as well as the target web application server. Thus, it is assumed that the administrator has run the AccessMatrix Installer to install the AccessMatrix UAM Server. Start the AccessMatrix UAM Server if it has not been started.

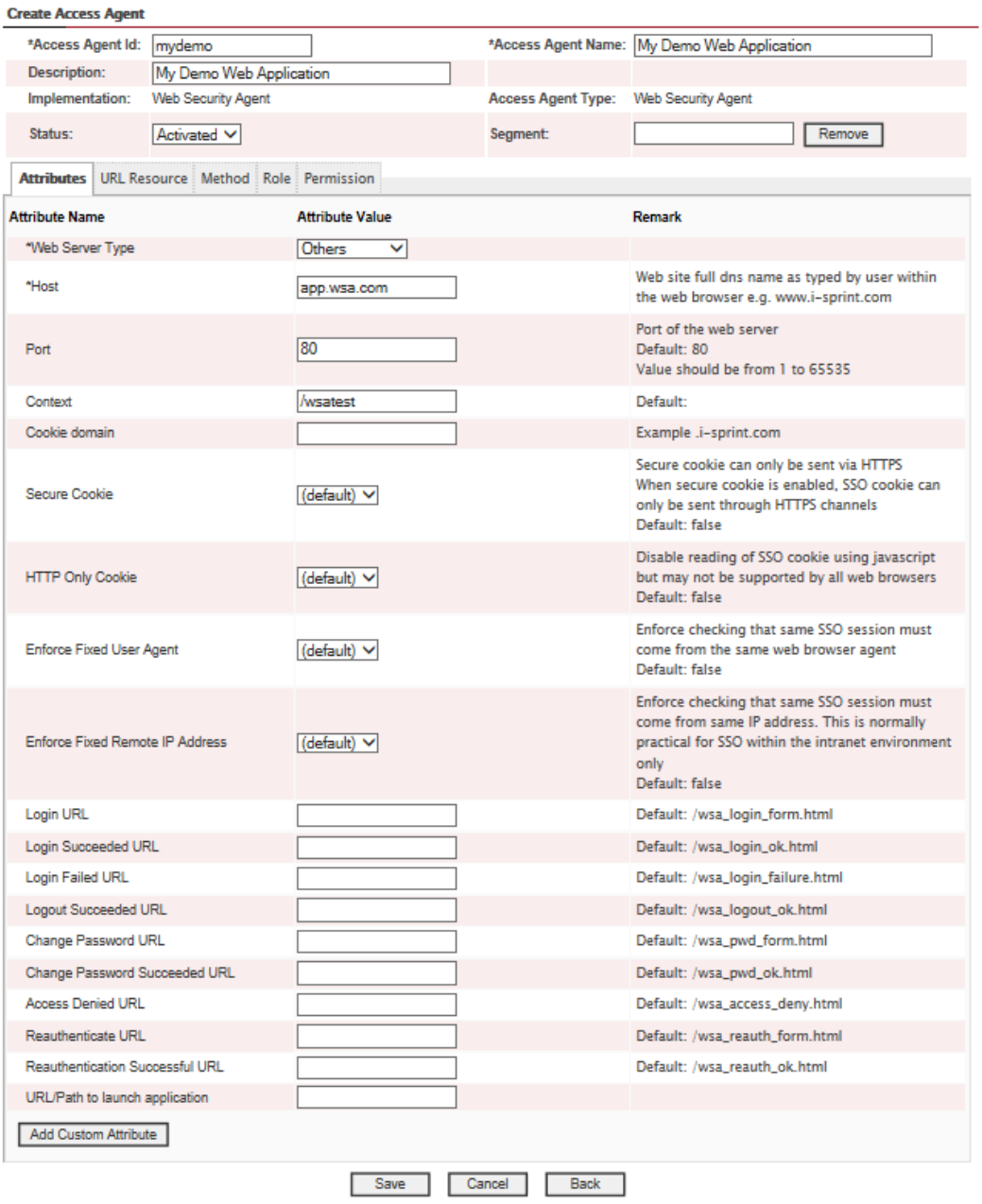
- WSA RP (Java) Sample Application
- WSA RP (Native) Sample Application

5.1. WSA RP (Java) Sample Application

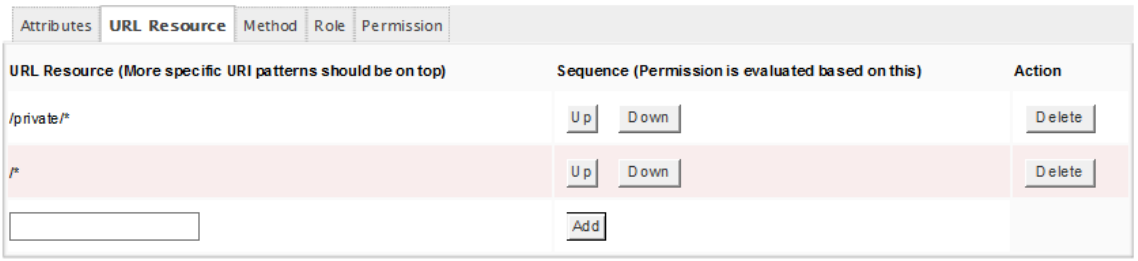
Create and Configure a New UAM Web Application in Admin Console

1. Navigate to **Access Agent Page** by selecting **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.
2. Create a Web Security Agent called "mydemo" for target web application context "wsatest" with the following information, and prefix all the URL fields with the same context as shown in following figure:
    - **Host**: "app.wsa.com"
    - **Port**: "80"

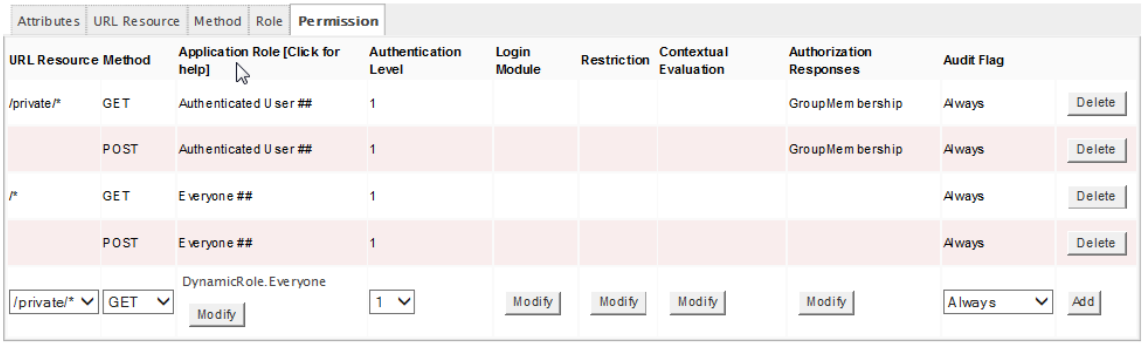
- **Context**: "/wsatest"



3. Click on the **URL Resource** tab to create the following URLs:



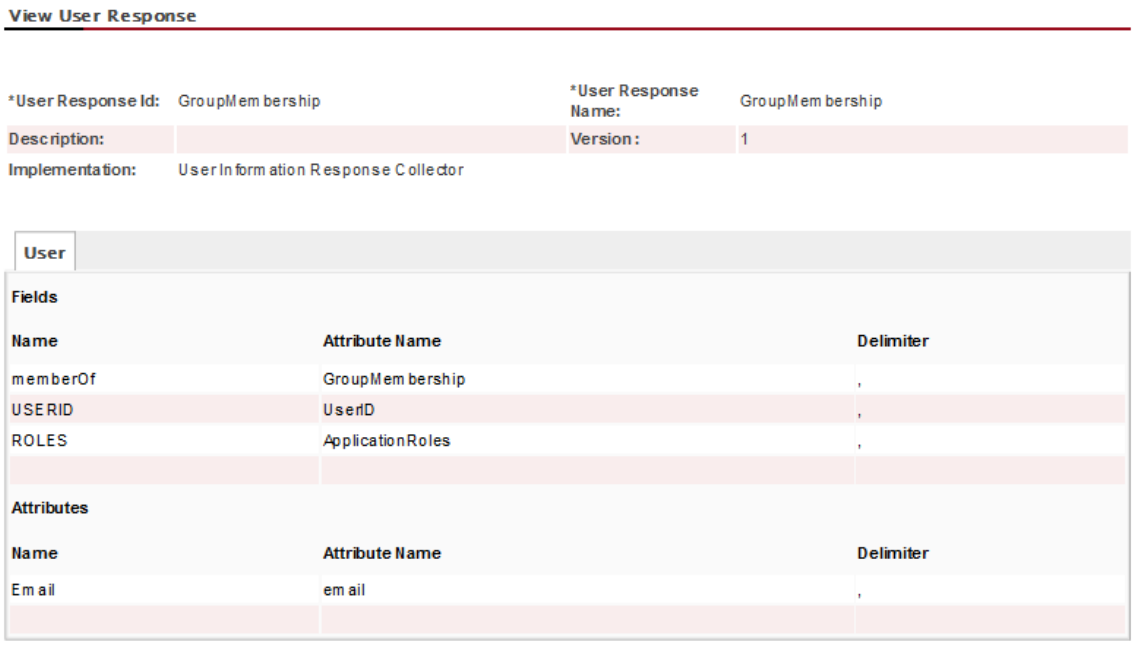
4. Click on the **Permission** tab to create the following permissions:



With the above permission configuration, WSA Reverse Proxy will inject a response configuration called "GroupMembership" into HTTP request headers at "/private/*" URL.

5. Save the changes.

6. Navigate to the **User Response Page** by selecting **Administration Dashboard > Configuration > Messaging & Response > User Response**. Configure the user response by editing the "GroupMembership" to map the HTTP header to the user attribute. These HTTP headers used for the mapping can be found in the **index.jsp** file.



Deploy WSA Reverse Proxy Sample Application to Tomcat

For this demonstration, the WSA Reverse Proxy sample application will be deployed as a target web application context mentioned in the previous section. Then, the target web application server will be configured to refer to the deployed target web application context accordingly.

1. Navigate to the bundled Tomcat's web application folder, i.e. *<ACMATRIX_ROOT>\tomcat\webapps* and create a new sub-folder called *wsatest*.
2. Extract the content of *<WSARP_PATH>/sample_wsa_app/wsa* folder and copy to the *wsatest* folder.
3. By default, the user will be redirected to the WSA login successful page defined in the system after being authenticated or reauthenticated successfully. In order to redirect the user to the target URL instead of the WSA login successful page, the administrator is required to edit the **wsa_login_ok.html** and **wsa_reauth_ok.html** files and uncomment the statement location.href=wsaTargetUrl, as shown in the following example:

   HTML

   ```
   if (wsaTargetUrl != null && wsaTargetUrl != "-1" &&
   wsaTargetUrl != "NULL") {
             location.href=wsaTargetUrl;
   }
   ```

4. Save the changes.
5. Restart the application server.

Test the Sample Application Deployment

Upon completion of the above configuration, the administrator is advised to perform a simple test to verify if the configuration is properly done. To do so, follow the steps below:

1. Open a web browser and visit the following URL: *http://server.wsa.com/wsatest/private/index.jsp* If WSA Reverse Proxy has been started and initialized successfully, the user will be redirected to the WSA login URL specified.
2. Login using the following user Id and password:
   - **User ID:** "demouser"
   - **Password:** "password"
3. Upon successful login, the landing page will display. Click on the continue link to access the **index.jsp** page. The user should be able to see the user Id printed out. Check the content of **index.jsp** to see how the request header is read.
4. The user may perform more tests for the HTTP request header injection, group membership, as well as user application roles. This is done by logging in to Admin Console and editing the response configuration for "GroupMembership" accordingly. For example, the user may add the new attribute called "EMAIL" in the "GroupMembership" response configuration.

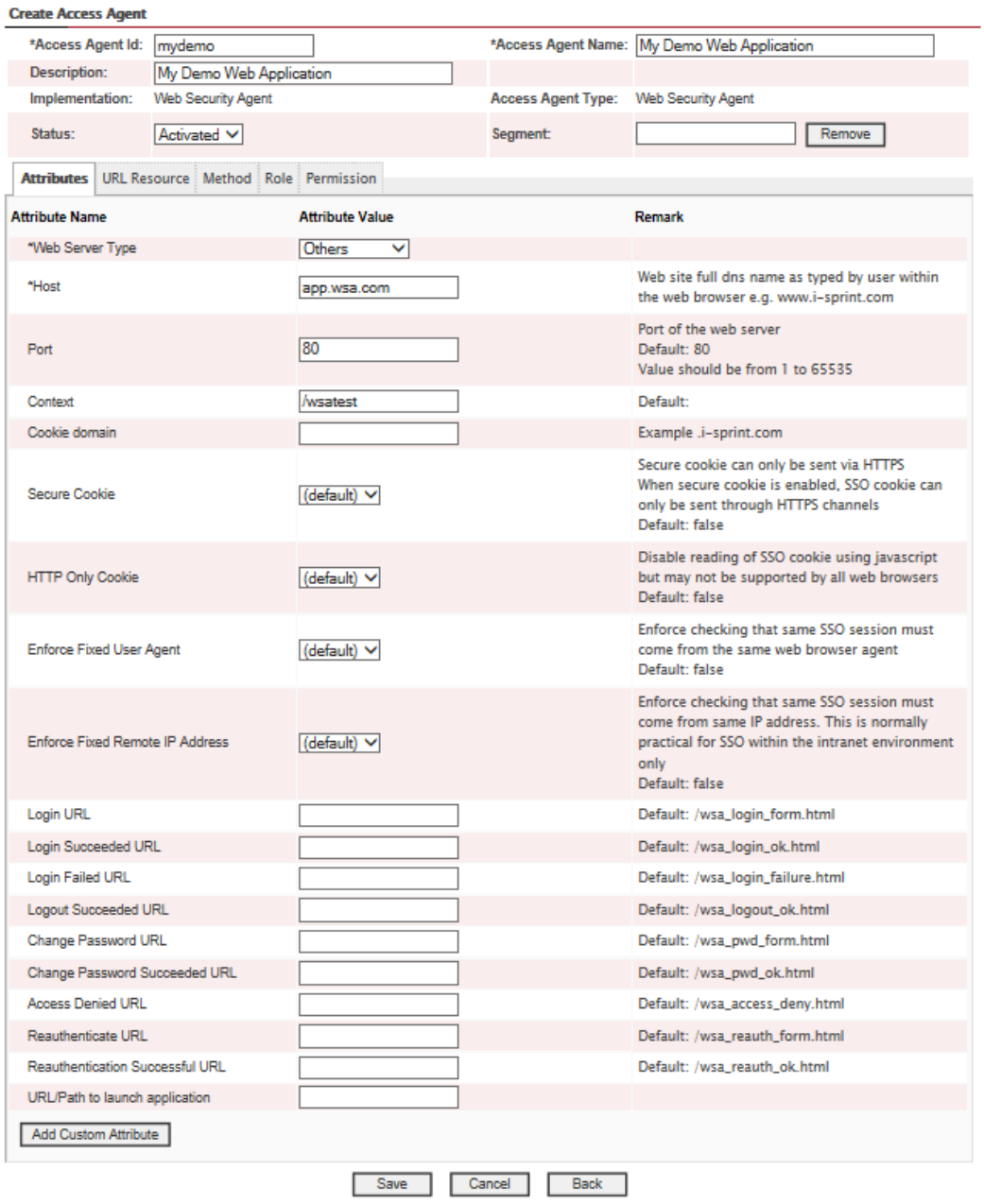
5. Having saved the above changes, edit the relevant user account, which in this case "demouser", to populate a dummy value for the "Email" user attribute, and edit the **index.jsp** to print out the EMAIL HTTP request header.

5.2. WSA RP (Native) Sample Application

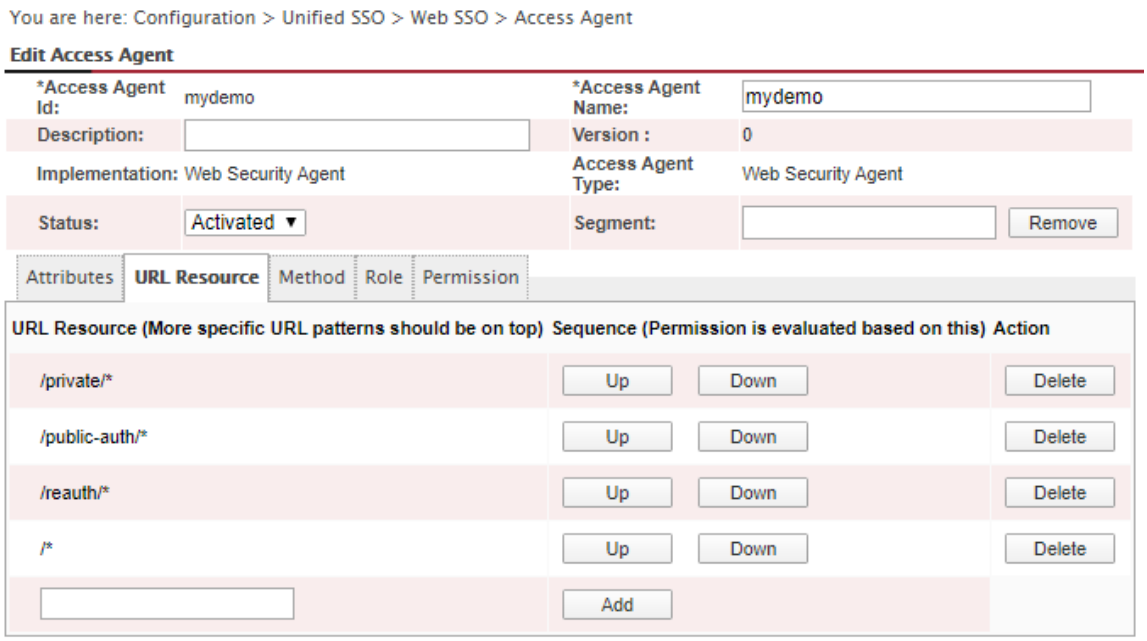
Create and Configure a New UAM Web Application in Admin Console

1. Navigate to **Access Agent Page** by selecting **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.
2. Create a Web Security Agent called "mydemo" for target web application context "wsatest" with the following information, and prefix all the URL fields with the same context as shown in following figure:
    - **Web Server Type:** "Others"
    - **Host: "**app.wsa.com"
    - **Port:** "80"

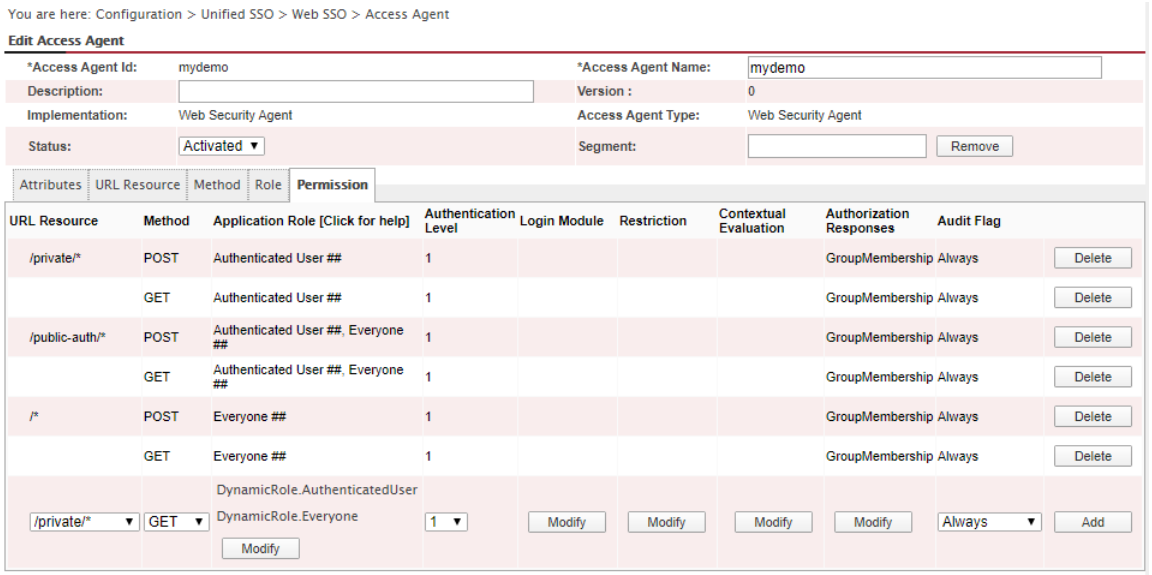
- **Context:** "/wsatest"



3. Click on the **URL Resource** tab to create the following URLs:



4. Click on the **Permission** tab to create the following permissions:

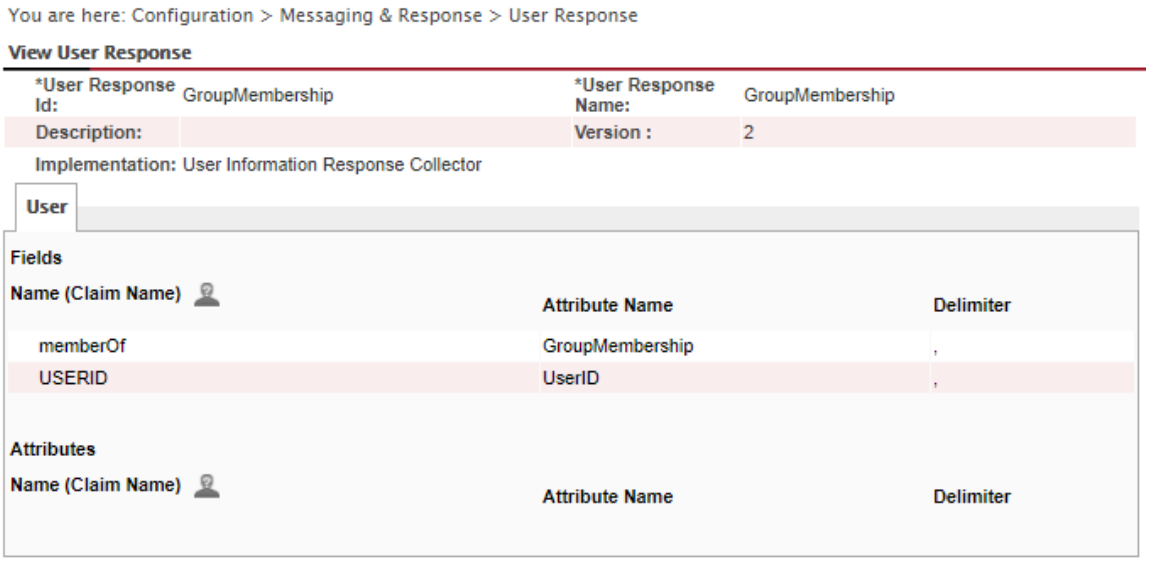


With the above permission configurations, WSA Reverse Proxy will inject a response configuration called "GroupMembership" into HTTP request headers at the "/private/*" URL.

5. Save the changes.

6. Navigate to the **User Response Page** by selecting **Administration Dashboard > Configuration > Messaging & Response > User Response**. Configure the user response by editing the "GroupMembership" to map the HTTP header to the user attribute. These HTTP headers used

for the mapping can be found in the **index.jsp** file.



Deploy WSA Reverse Proxy Sample Application to Tomcat

For this demonstration, the WSA Reverse Proxy sample application will be deployed as a target web application mentioned in the previous section. Then, the target web application server will be configured to refer to the deployed target web application accordingly.

1. Navigate to the bundled Tomcat's web application folder, i.e. *<ACMATRIX_ROOT>\tomcat\webapps* and create a new sub-folder called *wsatest*.
2. Extract the contents of *<WSARP_PATH>\wsatest* folder and copy them to *wsatest* folder.
3. Restart the application server.
4. Since the sample application is deployed in AccessMatrix Tomcat server, add the port to wsaApp in *<WSARP_PATH>/wsa/conf/wsarp.conf* as shown below:

   Code

   ```
   Define wsaApp http://app.wsa.com:8080/wsatest/
   ```

5. Save the changes and restart the WSA Reverse Proxy server.

Test the Sample Application Deployment

Upon completion of the above configuration, the administrator is advised to perform a simple test to verify if the configuration is properly done. To do so, follow the steps below:

1. Open a web browser and visit the following URL: *http://server.wsa.com/wsatest/private*.
   If WSA Reverse Proxy has been started and initialized successfully, you will be redirected to the specified WSA login URL.
2. Login using the following user id and password:
   - **User ID**: "demouser"
   - **Password**: "password"
3. Upon successful login, you will be prompted to change demouser's password. (By default, **Force Change Password** is set to "Yes" in Admin Console.)
4. Upon successful change of password, the landing page will be displayed. Click on the **Click to continue** link to access the **index.jsp** page. You should be able to see the user Id printed out (i.e. "demouser"). Check the content of **index.jsp** to see how the request header is read.
5. Perform more tests for the HTTP request header injection, group membership, as well as user application roles. This is done by logging in to Admin Console and editing the response configuration for "GroupMembership" accordingly. For example, add the new attribute called EMAIL into the GroupMembership response configuration.
6. Having saved the above changes, edit the relevant user account, which in this case is "demouser", to populate a dummy value for the "Email" user attribute, and edit the **index.jsp** to print out the EMAIL HTTP request header.

6. UAM WSA Reverse Proxy Web Services

WSA Reverse Proxy has a built-in web service that supports operations such as authentication, change password, log out, reauthentication and cross-domain Single Sign-On (SSO).

Authentication via Local WSA Reverse Proxy Web Service

Authentication request web service contract. A sample login page **wsa_login_form.html** is shipped within the installer.

URI Resource / Method:

HTTP POST to /context/wsa/wsaform

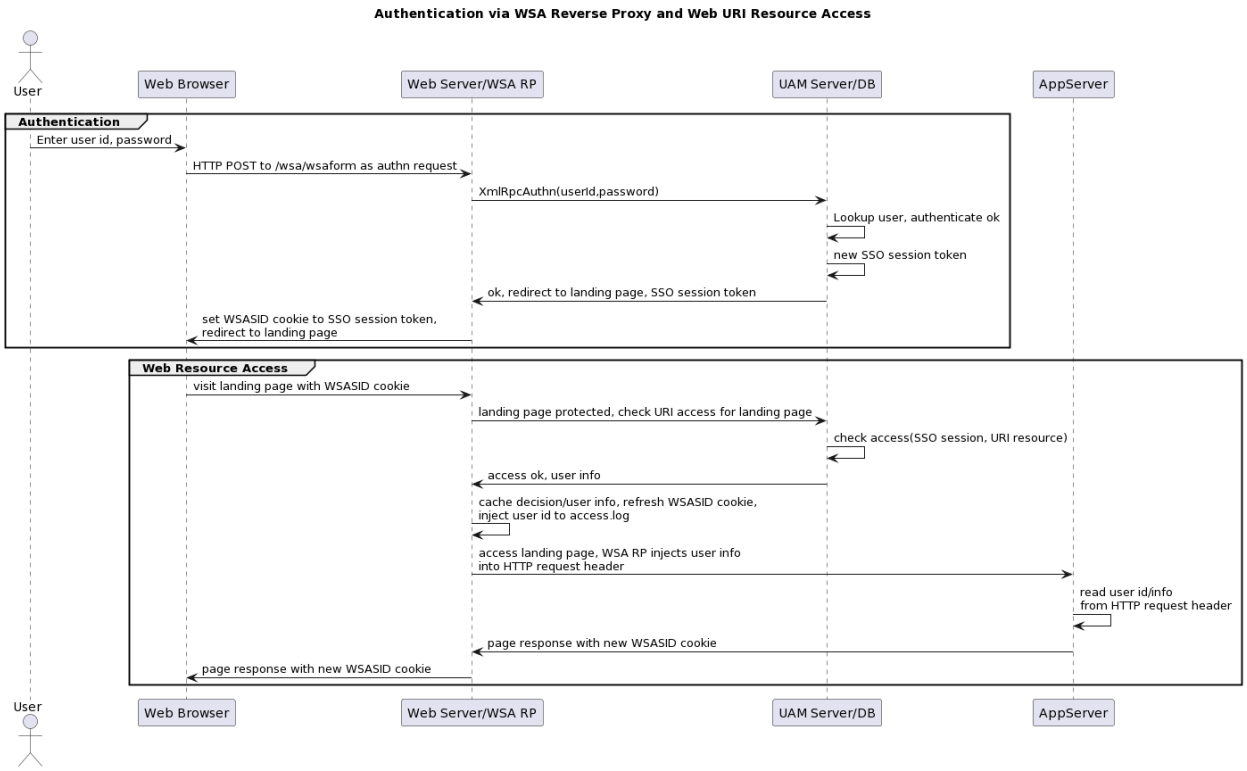
POST Parameters:

Key	Sample Value	Description
WSAPAMFIELD	40172	Authentication Realm Id
username	demouser	user Id
credential	password	user credential

Result:

1. UAM WSA Reverse Proxy will try to identify the relevant configured web security agent based on the host, port and context path of the requested URI.
2. If the application is not found, HTTP response code 406 will be issued.
3. If the application is found and authentication is successful, WSA Reverse Proxy will redirect the user to "login successful URI" as configured under the UAM application. A simple example page **wsa_login_ok.html** is shipped in the installer.
4. If authentication failed, WSA Reverse Proxy will redirect the user to "login failed URI" as configured under the UAM application. A simple example page **wsa_login_failure.html** is shipped in the installer.

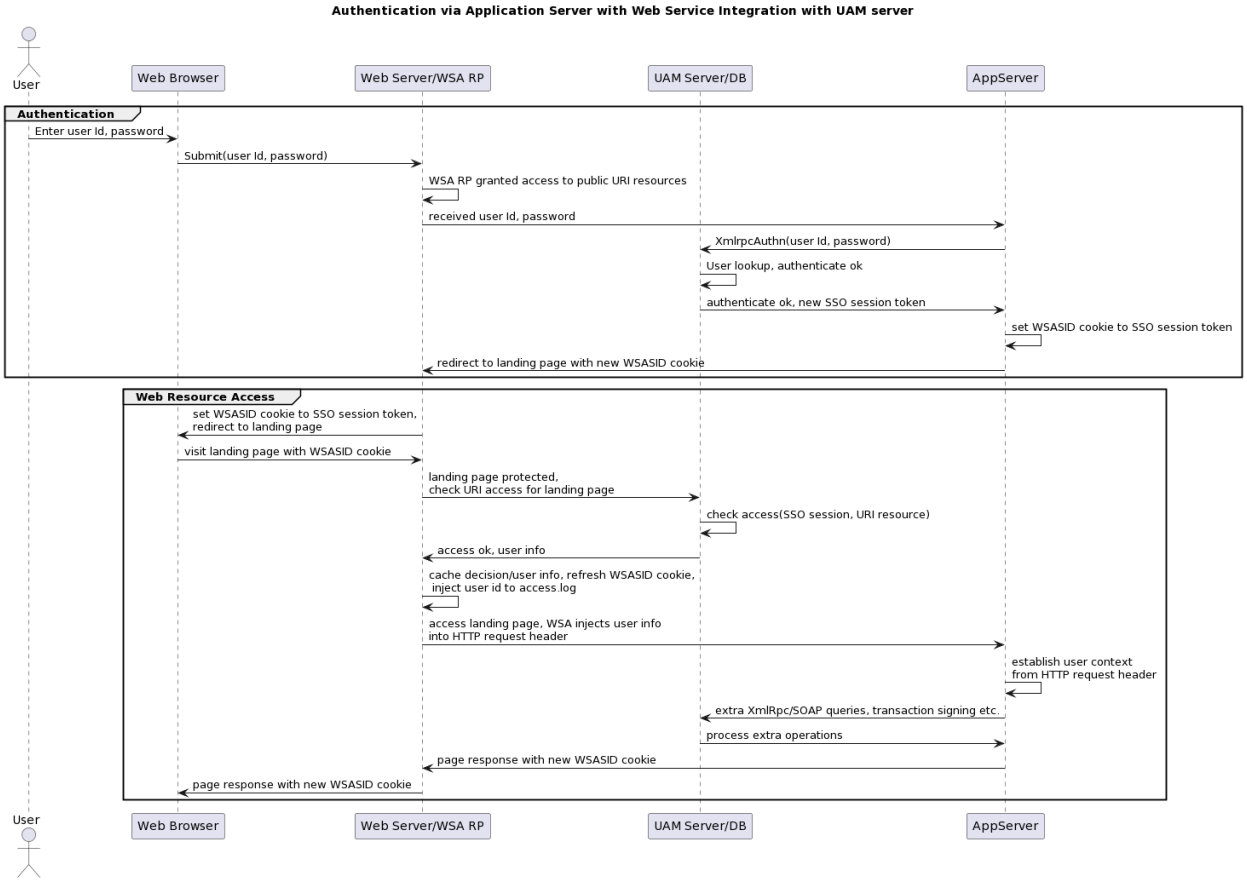
Sequence Diagram for Authentication via Local WSA RP Web Service



Authentication by a separate Identity Provider setting WSASID Cookie

The following sequence diagram assumes that authentication is performed via pages from the application server and integrates with UAM server via web services (XML-RPC or SOAP). Note that the authentication sequence is simplified, the actual authentication workflow could be more complex with 1FA or 2FA with multiple steps.

Sequence Diagram for Authentication via Application Server



WSA Reverse Proxy (Java)

Cross-Domain SSO where WSA Reverse Proxy acts as a Service Provider to consume SSO session

Cross-domain SSO applies to the situations when there is:

1. Absence of a Domain Name System (DNS) Server and web applications are accessed via IP address only.
2. Presence of DNS server but there are more than one DNS domains (cookie domains).

The cross-domain SSO scenario is similar to the Security Assertion Markup Language (SAML) Identity Provider (IdP)/Service Provider (SP) concept. For proprietary cross-domain SSO:

1. First of all, you need to set up an Identity Provider web site which normally is part of the employee portal or custom portal server. The roles of the IdP website besides providing authentication and setting the WSASID cookie value are to provide a list of UAM applications within the same cookie domain as well as different cookie domain. For the same cookie domain, there is no special handling required as WSASID cookie can be read by any UAM application within the same cookie domain.
2. For UAM applications (web links) in a different cookie domain, a special javascript / hidden form must be used to send the current WSASID cookie value to the corresponding WSA Reverse Proxy within a different cookie domain. This is similar to SAML Assertion Consumer Service URI:
3. For example, if the IdP URI is *http://idp.i-sprint.com* where the DNS domain is i-sprint.com, then the SP assertion consumer URI can be located at a separate domain *i-sprint.cn*:

Service Provider (Different Cookie Domain) Resource / Method:
http://i-sprint.cn/wsa/wsaform

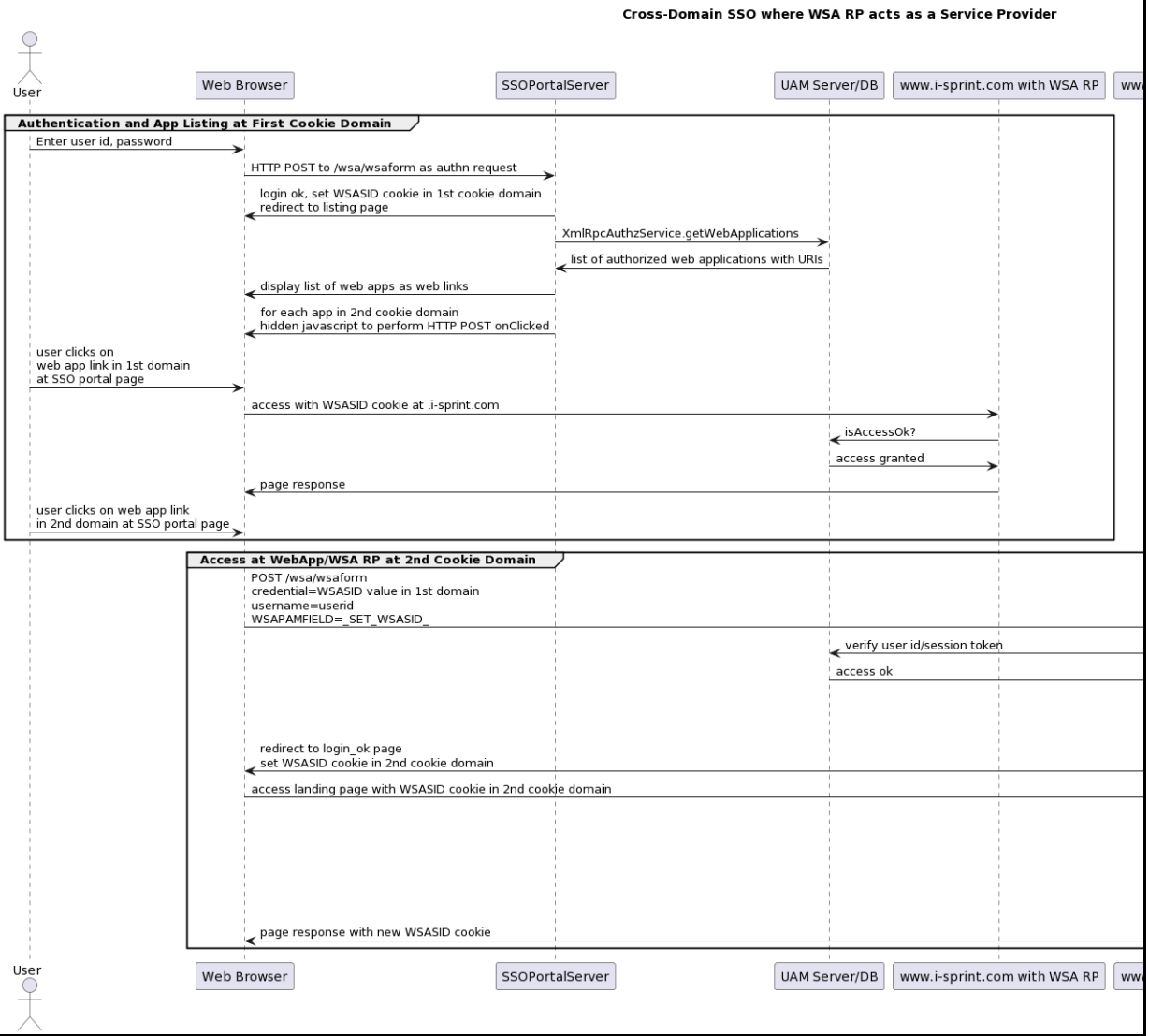
POST Parameters:

Key	Value	Description
username	user Id	
credential	SSO session token value	dynamic runtime value
WSAPAMFIELD	_SET_WSASID_	special hardcoded value

1. The SP that receives this hidden form **HTTP POST** must have WSA Reverse Proxy installed to protect it.
2. The SP WSA Reverse Proxy, when seeing this special WSAPAMFIELD value will verify the value of WSASID cookie, then set it within the local cookie domain, and redirect to login successful page configured for this UAM Application (Service Provider).
3. If a user jumps to another UAM application that does not have the WSASID cookie (i.e. a 3rd cookie domain), the WSA Reverse Proxy will redirect to the IdP assertion provider page and the whole process will repeat.

4. In the end, IdP must maintain its WSASID cookie and each SP must maintain its own WSASID cookies via its own WSA Reverse Proxy. Any idle timeout at any SP will redirect again to IdP assertion provider again.
5. There is a suggestion that IdP should have a much longer session idle timeout compared to the SP side. This can be achieved by exposing a new server-side API for assertion provider page to extend an already idle timeout session as long as it is still not expired. This is already part of the SAML standard so PS/partners are better off using SAML.
6. If single logout is required, there must be single logout provider page at IdP that first clears the IdP WSASID cookie then perform an **HTTP POST** to invoke /wsa/wsalogout of each SP's WSA Reverse Proxy to clear the SSO cookie in each cookie domain.

Sequence Diagram for Cross-Domain SSO



WSA Reverse Proxy (Native)	The Cross-Domain SSO section applies for the WSA Reverse Proxy (Java) implementation only.

Reauthentication to Higher Authentication Level

Authentication request web service contract. A sample login page **wsa_reauth_form.html** is shipped within the installer.

URI Resource / Method:

HTTP POST to /context/wsa/wsareauth

POST Parameters:

Key	Sample Value	Description
loginmodule	DefaultVascoTokenLogin	login module Id
credential	password	user credential

Result:

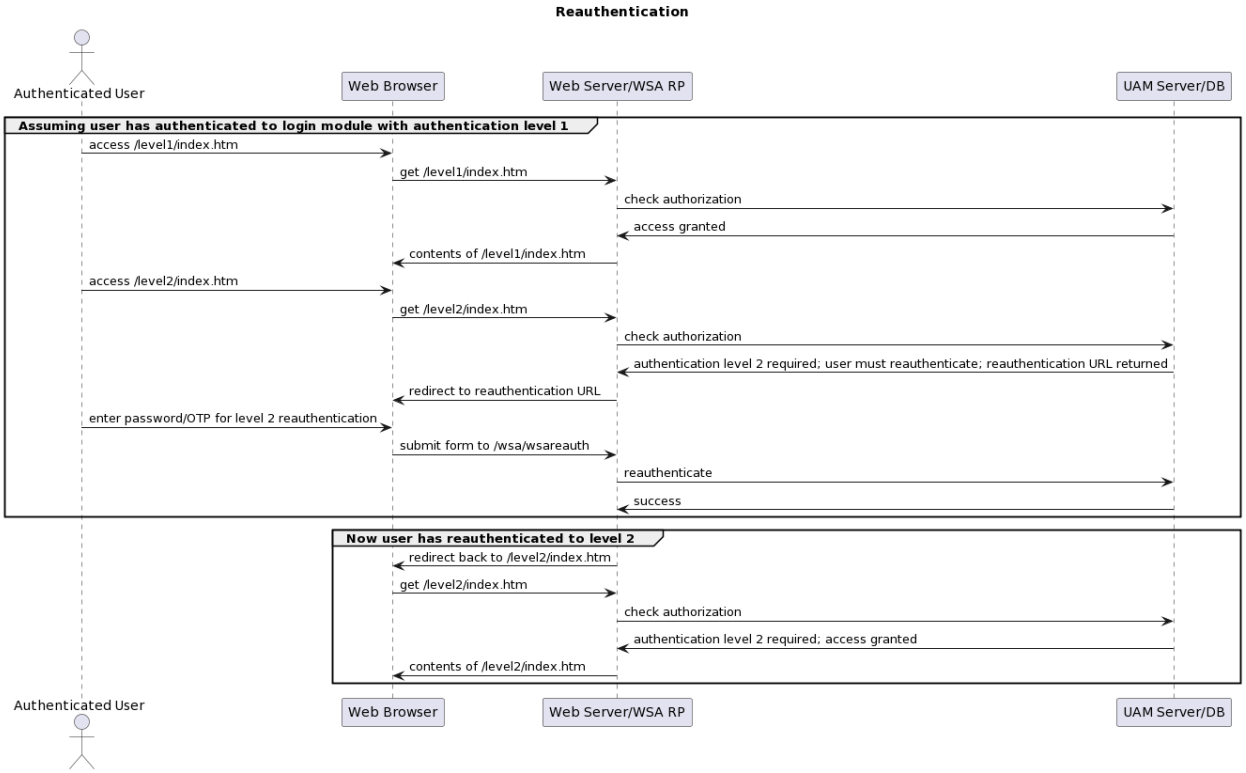
1. UAM WSA Reverse Proxy will automatically determine the UAM application Id from the host/port/context of the URI resource.
2. If the application is not found, the HTTP response code 406 will be issued.
3. If the user is not authenticated (i.e. WSASID cookie does not contain a valid authenticated session), the user will be redirected to the login page as configured within the UAM application.
4. If the application is found and reauthentication is successful, then WSA Reverse Proxy will redirect the user to "reauthentication successful URI" as configured under the UAM application. A simple example page **wsa_reauth_ok.html** is shipped in the installer.
5. If reauthentication failed, WSA Reverse Proxy will redirect the user to "reauthentication failed URI" as configured under the UAM application. A simple example page **wsa_reauth_fail.html** is shipped in the installer.

With the UAM application configuration in the Admin Console, URL resource permissions may be configured with different required authentication levels. The authentication level is a value from 1 to 10 associated with a login module. It gives a measure of the strength or security level of the authentication. A higher authentication level indicates a more secure login module.

Consider this example configuration:

URL	Authentication level
/level1/*	1
/level2/*	2

Sequence Diagram for Reauthentication



Change Password

User's change password web service contract. A sample login page **wsa_pwd_form.html** is shipped within the installer.

URI Resource / Method:

HTTP POST to /context/wsa/wsapwd

POST Parameters:

Key	Sample Value	Description
WSA_OLDPWD	password1	current password

Key	Sample Value	Description
WSA_NEWPWD1	password2	new password
WSA_NEWPWD2	password2	confirm new password

Result:

If the change password is successful, the user will be redirected to change password success page configured under the UAM application.

Logout

An **HTTP GET** on /context/wsa/wsalogout with valid WSASID cookie will log out the current SSO session and delete the WSASID cookie. The user will be redirected back to login page configured in the UAM application.

7. Other Apache-based Web Server Support (Java)

> **Note**: The following sections apply for the WSA Reverse Proxy (Java) implementation only.

WSA Support for Oracle HTTP Server 12c

Oracle HTTP Server (OHS) is a web server based on the Apache HTTP Server, created by the Oracle Technology Network. To support WSA RP plugin for OHS 12c, the following configurations should be done:

Configure httpd.conf

The default Oracle HTTP Server log mode is not compatible with Apache Web Server log mode used by WSA. Thus, the log mode needs to be changed to the "apache" log mode supported by Oracle. Refer to Oracle Diagnostic Logging Directives for additional details.

1. On the OHS, open the **httpd.conf** file with an editor.
   Example location: *$DOMAIN_HOME/config/fmwconfig/components/OHS/instances/ohs1/httpd.conf/*.
2. Search for the OraLogMode directive to set the logging format.
3. Set it to apache to switch the logging mode to the Standard Apache log format. Default setting is odl-text.

```
# Directives to setup logging via ODL
OraLogDir "${ORACLE_INSTANCE}/servers/${COMPONENT_NAME}/logs"
#OraLogMode odl-text
OraLogMode apache
OraLogSeverity WARNING:32
OraLogRotationParams S 10:70
```

4. Search for the ErrorLog directive and uncomment it to use the Apache log type.

```
# Logging mode is set to odl-text mode by default.
# If you wish to use the apache type log instead then uncomment the
# ErrorLog and LogLevel lines below and set OraLogMode to apache
#
ErrorLog "${ORACLE_INSTANCE}/servers/${COMPONENT_NAME}/logs/error_log"
```

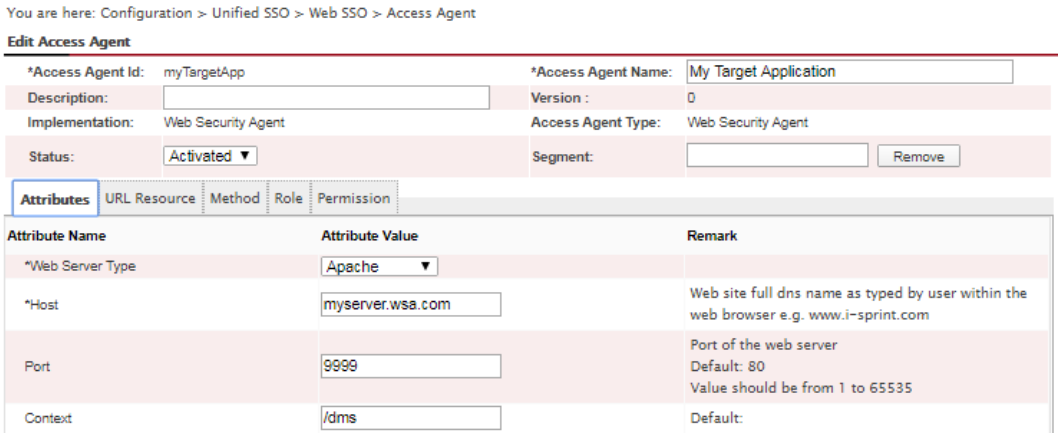
5. Save the changes.

Configure WSA Application

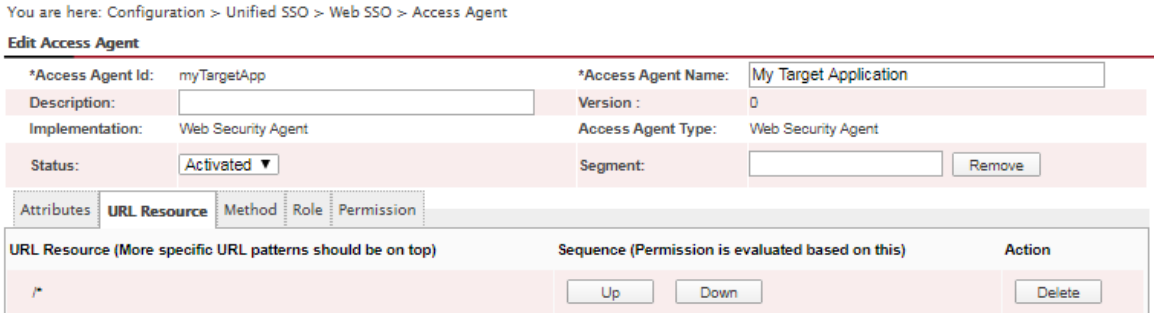
Oracle's Dynamic Monitoring Service (DMS) is an Oracle web application that requires local connection within the web server. When WSA is deployed, by default, it will also be protected by WSA. Thus, WSA

needs to be configured to add a WSA Application for the Dynamic Monitoring Service to bypass the protection to allow internal communication between Oracle Web Server and DMS.

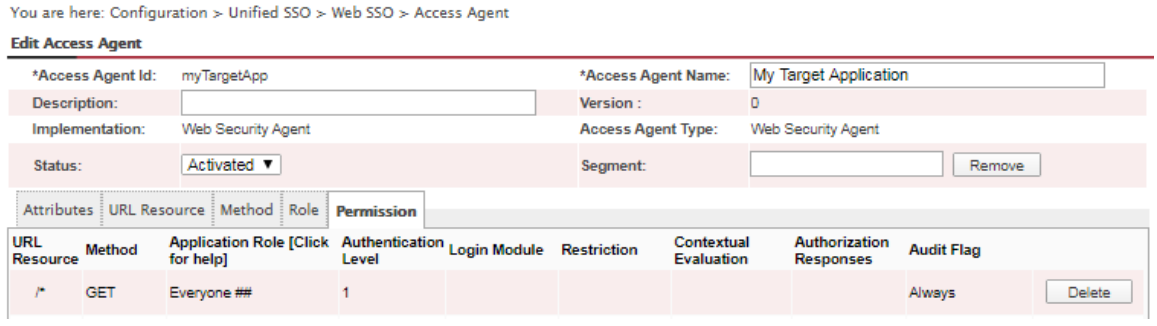
1. Navigate to the **Access Agent Page** by selecting **Administration Dashboard > Configuration > Unified SSO > Web SSO > Access Agent**.
2. Create a Web Security Agent called "myTargetApp" with the following information:
    - **Host**: IP Address/Host name of the target application, e.g. "myserver.wsa.com" .
    - **Port**: Port number of the target application, e.g. "9999".
    - **Context**: Context path of the target application, e.g. "/dms".



3. Click on the **URL Resource** tab to create the following URL:



4. Click on the **Permission** tab to create the following permission:



5. Save the changes.

8. Appendices

Refer to the following sections:

- WSA RP (Java) Appendices
- WSA RP (Native) Appendices

8.1. WSA RP (Java) Appendices

A. Troubleshooting and Monitoring

WSA Reverse Proxy supports the following logging for troubleshooting or monitoring.

A1. Startup

- WSA Reverse Proxy log integrates with Apache log in *httpd/logs/error_log*.

A2. Requests and User Activity

- For monitoring user activities, WSA Reverse Proxy injects user Id into standard web server access log. **Always enable standard web server access log** for WSA Reverse Proxy protected web servers to audit and monitor user activities.
- For troubleshooting request header injection, turn on debug logging for WSA Reverse Proxy core log in **wsarp.conf**. By default, all access denied and exceptional events will be logged within WSA Reverse Proxy core log with debug log level.
- WSA Reverse Proxy has strict checking on the host/port defined within Access Agent at Admin Console to represent the target web application. If accessed using a host/port combination not found at server-side, an HTTP response code 406 will be displayed. This is a common problem encountered during the initial setup of the WSA Reverse Proxy. Go to Admin Console to make sure the host/port defined is correct or use the correct host/port when accessing the web application.
- Under the exceptional situation, WSA Reverse Proxy will set a few common HTTP response codes, i.e. 406 if corresponding Access Agent is not found, 403 if access is denied, 503 if all AccessMatrix servers are not available (or internal error). Use the web server specific mapping services to map these to the specific application error handling pages if raw error codes are not to be seen by the users.
- Server-side log (**am.log**) can also be monitored by looking for the `authorizeForApp` keyword in the requests sent to AccessMatrix servers.

A3. Connection to AccessMatrix Server

Typically WSA Reverse Proxy is configured to connect to one primary AccessMatrix server and failover to secondary AccessMatrix server. WSA Reverse Proxy will log each of these events when:

- the primary server is down and WSA fails over to the secondary server
- the primary server is up again and WSA fails back to the primary server
- Sample logs

```
2013-05-22 12:44:23 [AWT-EventQueue-0] WARN - NOCODE-WSA xml rpc proxy factory failover to host#1 http://localhost:8888/wpe/xmlrpc
2013-05-22 12:44:40 [WSA-worker] WARN - NOCODE-WSA xml rpc proxy factory fail back to host#0 http://localhost:888/wpe/xmlrpc
```

B. Cookies used by WSA

Cookie	Description
WSAFAILMSG	This contains the WSA Fail Message. It is set by WSA when it detects error upon URL access to WSA protected resource. WSAFAILMSG is in this format: CATEGORY_ERROR__REFID Possible CATEGORY and ERROR are listed below. You may use the values for error handling (e.g. to display a certain message to the user or ask the user to log in again). REFID refers to the reference ID assigned to the error occurrence. The REFID is also printed in the log file for further troubleshooting to find more details about the error when required. **Categories:** AUTHENTICATION, AUTHORIZATION, SYSTEM **Errors:** AUTHENTICATION • VerificationFailed (user is not found, wrong password (invalid credentials)) • Other (for account status issues, e.g. expired, disabled, login failed denied by login policy (e.g. bad login threshold is reached), etc.) AUTHORIZATION • Other (for Authorization exceptions) SYSTEM • Other (Other exceptions, e.g. System exceptions) **Example of WSAFAILMSG and its corresponding log entry:** WSAFAILMSG=AUTHENTICATION_VerificationFailed__Oy2zuF8t 2020-04-16 16:03:03.415 [AM5WSA:INFO] RefId Oy2zuF8t -> class=com.isprint.am.dto.exception.AuthenticationException$VerificationFailed&message=Wrong password

Cookie	Description
	&code=10020&extendedCode=0&msgArg=SystemPasswordBasic&param.badLoginCount=1 &param.badLoginCountForChgPwd=0&param.maxBadLoginCount=3&maxBadLoginCount=3 &badLoginCount=1&lastTry=false&badLoginCountForChgPwd=0 WSAFAILMSG=AUTHENTICATION_Other__KFoeK0Iq 2020-04-16 16:19:06.815 [AM5WSA:INFO] RefId KFoeK0Iq -> class=com.isprint.am.dto.exception.AuthenticationException$DeniedByPolicyMaxFailedAttemptsReached&message=Max. Failed attempt is reached, badLoginCount=3, maxBadLogin=3&code=10403&extendedCode=0&msgArg=Max. Failed attempt is reached, badLoginCount=3, maxBadLogin=3&badLoginCount=3
WSAPAM	This contains the default Pluggable Authentication Module (PAM) for authentication.
WSAREAUTH	This contains the required authentication level. This is set by WSA when a user is required to reauthenticate to a higher level.
WSATAR	The Target URL. It is also the previous URL that the user tried to access so that WSA can redirect the user after authentication.
WSASID	The SSO Session ID. This is used by WSA to verify user identity.
MSG	This is similar to WSAFAILMSG.

8.2. WSA RP (Native) Appendices

A. Troubleshooting and Monitoring

WSA Reverse Proxy supports the following logging for troubleshooting or monitoring.

A1. Startup

- WSA Reverse Proxy log integrates with Apache log in *httpd/logs/error_log*.

A2. Requests and User Activity

- For monitoring user activities, WSA Reverse Proxy injects user Id into standard web server access log. **Always enable standard web server access log** for WSA Reverse Proxy protected web servers to audit and monitor user activities.
- For troubleshooting request header injection, turn on debug logging for WSA Reverse Proxy core log in **wsarp.conf**. By default, all access denied and exceptional events will be logged within WSA Reverse Proxy core log with debug log level.
- WSA Reverse Proxy has strict checking on the host/port defined within Access Agent at Admin Console to represent the target web application. If accessed using a host/port combination not found at server-side, an HTTP response code 406 will be displayed. This is a common problem encountered during the initial setup of the WSA Reverse Proxy. Go to Admin Console to make sure the host/port defined is correct or use the correct host/port when accessing the web application.
- Under the exceptional situation, WSA Reverse Proxy will set a few common HTTP response codes, i.e. 406 if corresponding Access Agent is not found, 403 if access is denied, 503 if all AccessMatrix servers are not available (or internal error). Use the web server specific mapping services to map these to the specific application error handling pages if raw error codes are not to be seen by the users.
- Server-side log (**am.log**) can also be monitored by looking for authorizeForApp keyword in the requests sent to AccessMatrix servers.

B. Cookies used by WSA

<table>
<tr><th>Cookie</th><th>Description</th></tr>
<tr><td>WSAFAILMSG</td><td>
This contains the WSA Fail Message. It is set by WSA when it detects error upon URL access to WSA protected resource. WSAFAILMSG is in this format:
CATEGORY_ERROR__REFID

Possible CATEGORY and ERROR are listed below. You may use the values for error handling (e.g. to display a certain message to the user or ask the user to log in again).

REFID refers to the reference ID assigned to the error occurrence. The REFID is also printed in the log file for further troubleshooting to find more details about the error when required.

Categories:
AUTHENTICATION, AUTHORIZATION, SYSTEM

Errors:
AUTHENTICATION
VerificationFailed (user is not found, wrong password (invalid credentials))
Other (for account status issues, e.g. expired, disabled, login failed denied by login policy (e.g. bad login threshold is reached), etc.)
AUTHORIZATION
Other (for Authorization exceptions)
SYSTEM
Other (Other exceptions, e.g. System exceptions)
Example of WSAFAILMSG and its corresponding log entry:

WSAFAILMSG=AUTHENTICATION_VerificationFailed__Oy2zuF8t
2020-04-16 16:03:03.415 [AM5WSA:INFO] RefId Oy2zuF8t ->
class=com.isprint.am.dto.exception.AuthenticationException$VerificationFailed&message=Wrong password
&code=10020&extendedCode=0&msgArg=SystemPasswordBasic&param.badLoginCount=1
&param.badLoginCountForChgPwd=0&param.maxBadLoginCount=3&maxBadLoginCount=3
&badLoginCount=1&lastTry=false&badLoginCountForChgPwd=0

WSAFAILMSG=AUTHENTICATION_Other__KFoeK0Iq
2020-04-16 16:19:06.815 [AM5WSA:INFO] RefId KFoeK0Iq ->
class=com.isprint.am.dto.exception.AuthenticationException$DeniedByPolicyMaxFailedAttemptsReached&message=Max. Failed attempt is reached, badLoginCount=3, maxBadLogin=3&code=10403&extendedCode=0&msgArg=Max. Failed attempt is reached, badLoginCount=3, maxBadLogin=3&badLoginCount=3
</td></tr>
</table>

Cookie	Description
WSAPAM	Contain default Pluggable Authentication Module (PAM) for authentication.
WSAREAUTH	Contain required authentication level. This is set by WSA when a user is required to reauthenticate to a higher level.
WSATAR	Target URL. It is also the previous URL that the user tried to access so that WSA can redirect the user after authentication.
WSASID	SSO Session ID. This is used by WSA to verify user identity.